

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH  
TRƯỜNG ĐẠI HỌC BÁCH KHOA



# BÀI TẬP LỚN HỆ CƠ SỞ DỮ LIỆU

**Giảng viên hướng dẫn:** Dương Huỳnh Anh Đức

**Nhóm sinh viên thực hiện:**

- |                       |         |
|-----------------------|---------|
| 1. Trần Thiện Nhân    | 2412443 |
| 2. Trần Xuân Lộc      | 2411964 |
| 3. Phan Ngọc Xuân Lợi | 2411969 |
| 4. Phan Kỳ Nhân       | 2412432 |
| 5. Võ Hoài Nam        | 2412204 |

TP. Hồ Chí Minh, tháng 3/2026

# Mục lục

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>1</b> | <b>GIỚI THIỆU</b>                                      | <b>3</b>  |
| <b>2</b> | <b>HIỆN THỰC CƠ SỞ DỮ LIỆU</b>                         | <b>7</b>  |
| 2.1      | Phía rạp . . . . .                                     | 7         |
| 2.1.1    | Phạm vi hiện thực phía Rạp . . . . .                   | 7         |
| 2.1.2    | Các bảng dữ liệu chính . . . . .                       | 7         |
| 2.1.3    | Các bảng phụ trợ . . . . .                             | 8         |
| 2.1.4    | Ràng buộc dữ liệu chính . . . . .                      | 8         |
| 2.1.5    | Dữ liệu mẫu . . . . .                                  | 8         |
| 2.2      | Phía khách hàng . . . . .                              | 9         |
| 2.2.1    | Phạm vi hiện thực phía Khách hàng . . . . .            | 9         |
| 2.2.2    | Các bảng dữ liệu chính . . . . .                       | 10        |
| 2.2.3    | Các bảng hỗ trợ nghiệp vụ khách hàng . . . . .         | 10        |
| 2.2.4    | Mối quan hệ giữa các bảng . . . . .                    | 11        |
| 2.2.5    | Ràng buộc dữ liệu chính . . . . .                      | 12        |
| 2.2.6    | Dữ liệu mẫu . . . . .                                  | 12        |
| 2.3      | Nhận xét . . . . .                                     | 13        |
| <b>3</b> | <b>THỦ TỤC THÊM, SỬA, XÓA DỮ LIỆU</b>                  | <b>14</b> |
| 3.1      | Bảng EVENT . . . . .                                   | 14        |
| 3.1.1    | Lý do chọn bảng EVENT . . . . .                        | 14        |
| 3.1.2    | Thủ tục thêm sự kiện . . . . .                         | 14        |
| 3.1.3    | Thủ tục sửa sự kiện . . . . .                          | 15        |
| 3.1.4    | Thủ tục xóa sự kiện . . . . .                          | 16        |
| 3.1.5    | Nhận xét . . . . .                                     | 16        |
| 3.2      | Bảng PRODUCT . . . . .                                 | 17        |
| 3.2.1    | Lý do chọn bảng PRODUCT . . . . .                      | 17        |
| 3.2.2    | Thủ tục thêm sản phẩm . . . . .                        | 17        |
| 3.2.3    | Thủ tục sửa sản phẩm . . . . .                         | 18        |
| 3.2.4    | Thủ tục xóa sản phẩm . . . . .                         | 19        |
| 3.2.5    | Nhận xét . . . . .                                     | 20        |
| <b>4</b> | <b>TRIGGER</b>                                         | <b>21</b> |
| 4.1      | Trigger trên bảng SESSION . . . . .                    | 21        |
| 4.1.1    | Trigger kiểm tra thời gian phiên sự kiện . . . . .     | 21        |
| 4.1.2    | Trigger cập nhật thuộc tính dẫn xuất . . . . .         | 22        |
| 4.1.3    | Nhận xét . . . . .                                     | 23        |
| 4.2      | Trigger phía Khách hàng . . . . .                      | 23        |
| 4.2.1    | Trigger kiểm tra điều kiện đánh giá sản phẩm . . . . . | 24        |

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| 4.2.2    | Các trigger hỗ trợ cho nghiệp vụ đánh giá . . . . .                | 25        |
| 4.2.3    | Trigger cập nhật tổng tiền đơn hàng . . . . .                      | 26        |
| 4.2.4    | Trigger hỗ trợ cập nhật tổng tiền khi áp dụng voucher . . . . .    | 27        |
| 4.2.5    | Trigger kiểm tra điều kiện áp dụng voucher . . . . .               | 28        |
| 4.2.6    | Trigger cập nhật điểm hiện tại của khách hàng thành viên . . . . . | 30        |
| 4.2.7    | Các trigger hỗ trợ khác trong phân hệ Khách hàng . . . . .         | 32        |
| 4.2.8    | Nhận xét . . . . .                                                 | 32        |
| <b>5</b> | <b>THỦ TỤC TRUY VẤN DỮ LIỆU</b>                                    | <b>33</b> |
| 5.1      | Các thủ tục truy vấn của phân hệ sự kiện . . . . .                 | 33        |
| 5.1.1    | Thủ tục hiển thị danh sách sự kiện . . . . .                       | 33        |
| 5.1.2    | Thủ tục thống kê sự kiện theo nhà tổ chức . . . . .                | 34        |
| 5.2      | Các thủ tục truy vấn của phân hệ Khách hàng . . . . .              | 35        |
| 5.2.1    | Thủ tục thống kê doanh thu sản phẩm . . . . .                      | 35        |
| <b>6</b> | <b>HÀM</b>                                                         | <b>39</b> |
| 6.1      | Các hàm của phân hệ sự kiện . . . . .                              | 39        |
| 6.1.1    | Hàm tính tổng phí thuê đã xác nhận . . . . .                       | 39        |
| 6.1.2    | Hàm đánh giá mức độ đáp ứng sức chứa . . . . .                     | 40        |
| 6.2      | Nhận xét . . . . .                                                 | 42        |
| 6.3      | Các hàm của phân hệ Khách hàng . . . . .                           | 42        |
| 6.3.1    | Hàm tính giá trị ròng của khách hàng . . . . .                     | 42        |
| 6.3.2    | Nhận xét . . . . .                                                 | 45        |
| <b>7</b> | <b>HIỆN THỰC ỨNG DỤNG</b>                                          | <b>46</b> |
| 7.1      | Tổng quan giao diện phía Rạp . . . . .                             | 46        |
| 7.1.1    | Giao diện quản lý sự kiện . . . . .                                | 46        |
| 7.1.2    | Giao diện tổng hợp nhà tổ chức . . . . .                           | 47        |
| 7.1.3    | Giao diện phân tích sự kiện . . . . .                              | 47        |
| 7.2      | Hiện thực ứng dụng phía Khách hàng và Sản phẩm . . . . .           | 48        |
| 7.2.1    | Giao diện quản lý sản phẩm . . . . .                               | 48        |
| 7.2.2    | Giao diện thống kê sản phẩm . . . . .                              | 49        |
| 7.2.3    | Giao diện giá trị khách hàng . . . . .                             | 50        |
| 7.2.4    | Nhận xét . . . . .                                                 | 50        |
| <b>8</b> | <b>TESTING VÀ DEMO</b>                                             | <b>51</b> |
| 8.1      | Mục tiêu kiểm thử . . . . .                                        | 51        |
| 8.2      | Kiểm thử bằng câu lệnh SQL . . . . .                               | 51        |
| 8.3      | Kiểm thử thông qua giao diện ứng dụng . . . . .                    | 53        |
| <b>9</b> | <b>KẾT QUẢ VÀ KẾT LUẬN</b>                                         | <b>54</b> |
| 9.1      | Kết quả . . . . .                                                  | 54        |
| 9.2      | Kết luận . . . . .                                                 | 55        |

# CHƯƠNG 1. GIỚI THIỆU

## Tổng quan đề tài

Trong bối cảnh công nghệ thông tin ngày càng phát triển, việc ứng dụng hệ thống phần mềm vào công tác quản lý đã trở thành nhu cầu cần thiết trong nhiều lĩnh vực. Đối với lĩnh vực giải trí, đặc biệt là hoạt động kinh doanh rạp chiếu phim, khối lượng dữ liệu phát sinh hằng ngày tương đối lớn và có mối liên hệ chặt chẽ với nhau. Các dữ liệu này bao gồm thông tin phim, thể loại phim, phòng chiếu, ghế ngồi, suất chiếu, khách hàng, nhân viên, vé, hóa đơn và doanh thu.

Nếu việc quản lý các nghiệp vụ trên được thực hiện thủ công bằng sổ sách hoặc các file lưu trữ rời rạc, hệ thống sẽ dễ gặp phải nhiều hạn chế như dữ liệu không đồng bộ, khó kiểm tra tình trạng ghế trống, mất thời gian khi tra cứu lịch chiếu, dễ sai sót khi lập hóa đơn và khó thống kê doanh thu. Những vấn đề này ảnh hưởng trực tiếp đến hiệu quả vận hành của rạp cũng như chất lượng phục vụ khách hàng.

Vì vậy, việc xây dựng một hệ thống quản lý rạp chiếu phim dựa trên cơ sở dữ liệu là cần thiết. Hệ thống giúp tổ chức dữ liệu một cách khoa học, đảm bảo tính chính xác, nhất quán và hỗ trợ tự động hóa các nghiệp vụ quản lý. Thông qua đó, người quản lý có thể dễ dàng theo dõi hoạt động của rạp, nhân viên có thể xử lý công việc nhanh chóng hơn và khách hàng được phục vụ hiệu quả hơn.

Trong bài tập lớn số 2 của môn Hệ cơ sở dữ liệu, nhóm tập trung vào giai đoạn hiện thực hệ thống. Cụ thể, nhóm tiến hành xây dựng cơ sở dữ liệu trên hệ quản trị MS SQL Server hoặc MySQL, tạo dữ liệu mẫu, cài đặt các thủ tục, trigger, hàm và phát triển ứng dụng minh họa có kết nối trực tiếp với cơ sở dữ liệu.

## Lý do chọn đề tài

Nhóm lựa chọn đề tài “Hệ thống quản lý rạp chiếu phim” vì đây là một bài toán thực tế, gần gũi với đời sống và có tính ứng dụng cao. Hoạt động của một rạp chiếu phim bao gồm nhiều nghiệp vụ rõ ràng như quản lý phim, quản lý lịch chiếu, bán vé, quản lý khách hàng, quản lý nhân viên và thống kê doanh thu. Những nghiệp vụ này phù hợp để xây dựng thành một hệ thống cơ sở dữ liệu hoàn chỉnh.

Bên cạnh đó, đề tài có nhiều thực thể dữ liệu liên kết với nhau, phù hợp với nội dung môn học Hệ cơ sở dữ liệu. Khi thực hiện đề tài, nhóm có thể vận dụng các kiến thức đã học như thiết kế bảng dữ liệu, xác định khóa chính và khóa ngoại, thiết lập ràng buộc toàn vẹn, viết thủ tục lưu trữ, xây dựng trigger, viết hàm xử lý dữ liệu và kết nối cơ sở dữ liệu với ứng dụng.

Ngoài ra, hệ thống quản lý rạp chiếu phim có quy trình nghiệp vụ dễ hình dung, thuận lợi cho việc phân chia công việc giữa các thành viên trong nhóm. Mỗi thành viên có thể phụ trách một phần cụ thể như thiết kế bảng, viết thủ tục thêm/sửa/xóa, xây dựng trigger, viết hàm, tạo dữ liệu mẫu hoặc hiện thực giao diện ứng dụng.

## Mục tiêu đề tài

Mục tiêu của đề tài là xây dựng một hệ thống quản lý rạp chiếu phim ở mức cơ bản, đáp ứng các nghiệp vụ chính trong quá trình vận hành rạp. Hệ thống không chỉ dừng lại ở việc thiết kế cơ sở dữ liệu

mà còn hiện thực các chức năng xử lý dữ liệu và xây dựng ứng dụng minh họa kết nối trực tiếp với cơ sở dữ liệu.

Các mục tiêu cụ thể của đề tài gồm:

- Xây dựng cơ sở dữ liệu quan hệ cho hệ thống quản lý rạp chiếu phim.
- Tạo đầy đủ các bảng dữ liệu cần thiết như phim, thể loại, phòng chiếu, ghế, suất chiếu, khách hàng, nhân viên, vé và hóa đơn.
- Thiết lập khóa chính, khóa ngoại và các ràng buộc dữ liệu nhằm đảm bảo tính toàn vẹn của hệ thống.
- Tạo dữ liệu mẫu có ý nghĩa cho các bảng để phục vụ quá trình kiểm thử và demo.
- Xây dựng các thủ tục thêm, sửa, xóa dữ liệu cho một bảng chính trong hệ thống, có kiểm tra dữ liệu đầu vào và thông báo lỗi phù hợp.
- Xây dựng các trigger nhằm kiểm tra ràng buộc nghiệp vụ và tự động cập nhật các thuộc tính dẫn xuất.
- Xây dựng các thủ tục truy vấn dữ liệu phục vụ chức năng tìm kiếm, lọc, sắp xếp và thống kê.
- Xây dựng các hàm xử lý dữ liệu có tham số đầu vào, có kiểm tra điều kiện và thực hiện tính toán dựa trên dữ liệu trong cơ sở dữ liệu.
- Phát triển ứng dụng minh họa có khả năng kết nối với cơ sở dữ liệu và gọi các thủ tục, hàm đã xây dựng.
- Kiểm thử hệ thống thông qua các tình huống thêm, sửa, xóa, truy vấn và xử lý lỗi dữ liệu.

## Ý nghĩa thực tiễn của đề tài

Đề tài có ý nghĩa thực tiễn trong việc mô phỏng một hệ thống quản lý rạp chiếu phim có thể áp dụng vào môi trường thực tế. Đối với người quản lý, hệ thống giúp theo dõi tình trạng phim đang chiếu, lịch chiếu, số lượng vé bán ra, doanh thu theo từng khoảng thời gian và hiệu quả hoạt động của từng suất chiếu.

Đối với nhân viên rạp, hệ thống hỗ trợ thao tác nghiệp vụ nhanh chóng hơn như tra cứu suất chiếu, kiểm tra thông tin vé, cập nhật dữ liệu khách hàng, lập hóa đơn và xử lý các tình huống phát sinh. Việc sử dụng cơ sở dữ liệu giúp giảm thiểu sai sót so với quản lý thủ công, đồng thời đảm bảo dữ liệu được lưu trữ tập trung và nhất quán.

Đối với khách hàng, một hệ thống quản lý tốt sẽ góp phần nâng cao chất lượng phục vụ. Thông tin về phim, suất chiếu và tình trạng vé được xử lý chính xác hơn, từ đó giúp khách hàng tiết kiệm thời gian và có trải nghiệm tốt hơn khi sử dụng dịch vụ tại rạp.

Ngoài ý nghĩa thực tế, đề tài còn giúp nhóm củng cố kiến thức môn học. Thông qua quá trình hiện thực hệ thống, nhóm hiểu rõ hơn cách vận dụng SQL, ràng buộc dữ liệu, thủ tục lưu trữ, trigger, hàm và kết nối ứng dụng với cơ sở dữ liệu trong một bài toán cụ thể.

## Phạm vi thực hiện

Trong phạm vi bài tập lớn số 2, nhóm tập trung vào giai đoạn hiện thực và vận hành hệ thống quản lý rạp chiếu phim. Nội dung thực hiện bao gồm:

- Hiện thực các bảng dữ liệu đã thiết kế ở bài tập lớn số 1.
- Cài đặt đầy đủ khóa chính, khóa ngoại và các ràng buộc dữ liệu cần thiết.
- Tạo dữ liệu mẫu cho các bảng, đảm bảo dữ liệu có ý nghĩa và phù hợp với nghiệp vụ.
- Viết thủ tục thêm, sửa, xóa dữ liệu cho một bảng trong hệ thống.
- Viết trigger kiểm tra ràng buộc nghiệp vụ và trigger cập nhật thuộc tính dẫn xuất.
- Viết thủ tục truy vấn dữ liệu có sử dụng điều kiện lọc, sắp xếp, nhóm dữ liệu và hàm tổng hợp.
- Viết hàm xử lý và tính toán dữ liệu dựa trên tham số đầu vào.
- Xây dựng ứng dụng minh họa có giao diện thêm, sửa, xóa, tìm kiếm, hiển thị danh sách và gọi các thủ tục hoặc hàm trong cơ sở dữ liệu.
- Thực hiện kiểm thử và demo các chức năng chính của hệ thống.

Do giới hạn về thời gian và phạm vi môn học, đề tài chưa tập trung vào các chức năng nâng cao như đặt vé trực tuyến qua website, thanh toán online, gửi email xác nhận vé, tích hợp mã QR, quản lý nhiều chi nhánh rạp hoặc phát triển ứng dụng di động hoàn chỉnh. Những chức năng này có thể được mở rộng trong các phiên bản phát triển tiếp theo.

## Công nghệ sử dụng

Để thực hiện đề tài, nhóm sử dụng các công nghệ và công cụ sau:

- Hệ quản trị cơ sở dữ liệu: Microsoft SQL Server hoặc MySQL.
- Ngôn ngữ truy vấn và xử lý dữ liệu: SQL.
- Công cụ quản lý cơ sở dữ liệu: SQL Server Management Studio hoặc MySQL Workbench.
- Ngôn ngữ lập trình ứng dụng: C#, Java hoặc Python.
- Môi trường phát triển giao diện: Windows Forms, Web Application hoặc công cụ lập trình phù hợp.
- Công cụ soạn thảo báo cáo: LaTeX.

Việc lựa chọn các công nghệ trên nhằm đảm bảo hệ thống có thể hiện thực đầy đủ các yêu cầu của bài tập lớn, đồng thời giúp nhóm làm quen với quy trình xây dựng một hệ thống cơ sở dữ liệu có kết nối với ứng dụng thực tế.

## Bố cục báo cáo

Báo cáo được trình bày theo các nội dung chính sau:

- Chương 1: Giới thiệu tổng quan về đề tài, lý do chọn đề tài, mục tiêu, phạm vi và công nghệ sử

dụng.

- Chương 2: Hiện thực cơ sở dữ liệu, bao gồm tạo bảng, ràng buộc dữ liệu và dữ liệu mẫu.
- Chương 3: Trình bày các thủ tục thêm, sửa, xóa dữ liệu.
- Chương 4: Trình bày các trigger dùng để kiểm tra ràng buộc nghiệp vụ và cập nhật dữ liệu tự động.
- Chương 5: Trình bày các thủ tục truy vấn dữ liệu.
- Chương 6: Trình bày các hàm xử lý và tính toán dữ liệu.
- Chương 7: Trình bày phần hiện thực ứng dụng kết nối với cơ sở dữ liệu.
- Chương 8: Trình bày quá trình testing và demo hệ thống.
- Chương 9: Tổng kết kết quả đạt được, hạn chế và hướng phát triển.

## Kết luận chương

Chương này đã trình bày tổng quan về đề tài “Hệ thống quản lý rạp chiếu phim”, bao gồm lý do lựa chọn đề tài, mục tiêu thực hiện, ý nghĩa thực tiễn, phạm vi triển khai và các công nghệ sử dụng. Đây là cơ sở để nhóm tiến hành hiện thực cơ sở dữ liệu, xây dựng các thủ tục xử lý, trigger, hàm và ứng dụng minh họa trong các chương tiếp theo.

Ở bài tập lớn số 1, nhóm đã tập trung vào giai đoạn phân tích và thiết kế hệ thống. Trong bài tập lớn số 2, nhóm tiếp tục phát triển đề tài ở giai đoạn hiện thực và vận hành, nhằm xây dựng một hệ thống có khả năng lưu trữ, xử lý, truy vấn dữ liệu và kết nối với ứng dụng thực tế.

## CHƯƠNG 2. HIỆN THỰC CƠ SỞ DỮ LIỆU

### 2.1 Phía rạp

#### 2.1.1 Phạm vi hiện thực phía Rạp

Phân hệ phía Rạp được xây dựng để quản lý các thông tin liên quan đến hoạt động vận hành rạp chiếu phim, bao gồm rạp, phòng chiếu, lịch chiếu phim và sự kiện được tổ chức tại rạp. Cơ sở dữ liệu của hệ thống có nhiều bảng phục vụ các nghiệp vụ khác nhau. Vì vậy, báo cáo không liệt kê toàn bộ các bảng trong phần này, mà chỉ trình bày những bảng có liên quan trực tiếp đến phần phía Rạp và các chức năng được hiện thực ở các chương sau.

#### 2.1.2 Các bảng dữ liệu chính

| Bảng      | Vai trò trong phân hệ phía Rạp                                                                                                                                                                                         |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CINEMA    | Lưu thông tin rạp chiếu như tên rạp, giờ hoạt động, loại rạp, trạng thái và địa chỉ. Đây là bảng dùng để quản lý các rạp trong hệ thống.                                                                               |
| ROOM      | Lưu thông tin phòng chiếu thuộc từng rạp, bao gồm loại phòng, hệ thống âm thanh, sức chứa, trạng thái kỹ thuật và sơ đồ ghế. Phòng chiếu được dùng để tổ chức suất chiếu phim hoặc phiên sự kiện.                      |
| MOVIE     | Lưu thông tin phim được chiếu tại rạp như tên phim, thời lượng, ngày phát hành, độ tuổi giới hạn và trạng thái phát hành. Thông tin thời lượng phim được dùng khi xác định khoảng thời gian của một suất chiếu.        |
| SHOWTIME  | Lưu lịch chiếu phim tại các phòng chiếu. Bảng này được dùng để kiểm tra lịch sử dụng phòng, đặc biệt khi cần đối chiếu với các phiên tổ chức sự kiện.                                                                  |
| ORGANIZER | Lưu thông tin nhà tổ chức sự kiện như loại nhà tổ chức, tên, số điện thoại, email và địa chỉ. Mỗi sự kiện được gắn với một nhà tổ chức cụ thể.                                                                         |
| CONTRACT  | Lưu thông tin hợp đồng giữa rạp và nhà tổ chức, bao gồm thời gian hiệu lực, giá trị hợp đồng, phương thức thanh toán và các điều khoản liên quan.                                                                      |
| EVENT     | Lưu thông tin chính của sự kiện, bao gồm tên sự kiện, mô tả, thời gian bắt đầu, thời gian kết thúc, quy mô dự kiến, ngân sách và trạng thái sự kiện. Đây là bảng trung tâm trong phần hiện thực phía Rạp.              |
| SESSION   | Lưu các phiên tổ chức cụ thể của sự kiện tại phòng chiếu, bao gồm thời gian bắt đầu, thời gian kết thúc, số khách tối đa, phí thuê và trạng thái phiên. Bảng này được dùng nhiều trong các trigger kiểm tra nghiệp vụ. |

**Bảng 2.1:** Các bảng chính trong phân hệ phía Rạp

### 2.1.3 Các bảng phụ trợ

Ngoài các bảng chính, cơ sở dữ liệu còn có một số bảng phụ trợ để mô tả đầy đủ hơn hoạt động của rạp. Các bảng `CINEMA_PHONE`, `ROOM_SUPPORTED_SCREEN` và `SEAT` lưu thêm thông tin về số điện thoại rạp, định dạng màn hình được hỗ trợ và sơ đồ ghế của phòng chiếu. Các bảng `GENRE`, `ACTOR`, `DIRECTOR` và các bảng liên kết với `MOVIE` dùng để mô tả thông tin phim. Nhóm bảng `PERSON`, `EMPLOYEE`, `SHIFT`, `WORK_ASSIGNMENT` và `SHOWTIME_ASSIGNMENT` dùng cho phần quản lý và phân công nhân sự.

Mã SQL đầy đủ để tạo bảng, tạo ràng buộc và thêm dữ liệu mẫu được nộp kèm trong file script SQL của nhóm.

### 2.1.4 Ràng buộc dữ liệu chính

Trong quá trình hiện thực cơ sở dữ liệu, các ràng buộc đơn giản được cài đặt trực tiếp trong câu lệnh tạo bảng bằng khóa chính, khóa ngoại, `NOT NULL`, `UNIQUE`, `CHECK` và giá trị mặc định. Cách làm này giúp đảm bảo dữ liệu được kiểm tra ngay tại tầng cơ sở dữ liệu và tránh sử dụng trigger cho những điều kiện có thể kiểm tra trực tiếp bằng ràng buộc bảng.

Đối với phân hệ phía Rạp, các ràng buộc chính gồm:

- Các bảng như `CINEMA`, `ROOM`, `MOVIE`, `SHOWTIME`, `ORGANIZER`, `CONTRACT`, `EVENT` và `SESSION` đều có khóa chính để định danh từng dòng dữ liệu.
- Các quan hệ giữa bảng được kiểm soát bằng khóa ngoại, chẳng hạn `ROOM` thuộc `CINEMA`, `SHOWTIME` gắn với `MOVIE` và `ROOM`, `EVENT` gắn với `ORGANIZER` và `CONTRACT`, còn `SESSION` gắn với `EVENT` và `ROOM`.
- Một số thuộc tính số như sức chứa phòng, thời lượng phim, quy mô dự kiến, ngân sách, số khách tối đa và phí thuê được kiểm tra để tránh nhận giá trị âm hoặc không hợp lệ.
- Các thuộc tính trạng thái như trạng thái rạp, trạng thái phòng, trạng thái sự kiện và trạng thái phiên được giới hạn trong các giá trị hợp lệ theo nghiệp vụ.

Những ràng buộc phức tạp hơn, chẳng hạn kiểm tra thời gian của phiên sự kiện có nằm trong thời gian diễn ra sự kiện hay không, được xử lý bằng trigger và sẽ được trình bày ở chương sau.

### 2.1.5 Dữ liệu mẫu

Dữ liệu mẫu được tạo cho các bảng trong phân hệ phía Rạp nhằm phục vụ việc kiểm thử ràng buộc, thủ tục, trigger và hàm. Các dữ liệu này bao gồm thông tin rạp, phòng chiếu, phim, lịch chiếu, nhà tổ chức, hợp đồng, sự kiện và các phiên tổ chức sự kiện.

Dữ liệu mẫu được thiết kế để có thể kiểm tra cả các trường hợp hợp lệ và không hợp lệ. Ví dụ, dữ liệu về `EVENT` và `SESSION` được dùng để kiểm tra thủ tục thêm, sửa, xóa sự kiện; trigger kiểm tra thời gian phiên sự kiện; trigger cập nhật số phiên đã xác nhận; cũng như các hàm tính toán liên quan đến phí thuê và mức độ đáp ứng sức chứa.

Toàn bộ script tạo cơ sở dữ liệu, tạo ràng buộc và thêm dữ liệu mẫu được nộp kèm trong file SQL của nhóm.

## 2.2 Phía khách hàng

### 2.2.1 Phạm vi hiện thực phía Khách hàng

Phân hệ phía Khách hàng được xây dựng nhằm quản lý các thông tin và nghiệp vụ liên quan đến khách hàng trong hệ thống rạp chiếu phim. Các nghiệp vụ chính bao gồm quản lý thông tin khách hàng, tài khoản khách hàng đăng ký, đơn hàng, chi tiết đơn hàng, vé điện tử, sản phẩm đi kèm, thanh toán, hoàn đổi, đánh giá, điểm tích lũy, voucher và yêu cầu dịch vụ tại ghế.

Trong hệ thống rạp chiếu phim, khách hàng là đối tượng trực tiếp thực hiện các giao dịch như đặt vé, mua sản phẩm, thanh toán hóa đơn, sử dụng voucher, tích điểm và gửi đánh giá sau khi sử dụng dịch vụ. Vì vậy, phân hệ phía Khách hàng đóng vai trò quan trọng trong việc lưu trữ lịch sử giao dịch, hỗ trợ chăm sóc khách hàng và cung cấp dữ liệu cho các chức năng thống kê, báo cáo doanh thu.

Cơ sở dữ liệu của hệ thống có nhiều bảng liên kết với nhau. Trong phần này, báo cáo chỉ tập trung trình bày các bảng có liên quan trực tiếp đến phân hệ Khách hàng và các chức năng được hiện thực ở những chương sau.

### 2.2.2 Các bảng dữ liệu chính

| Bảng                | Vai trò trong phân hệ phía Khách hàng                                                                                                                                                                                            |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CUSTOMER            | Lưu thông tin định danh khách hàng trong hệ thống. Bảng này liên kết với bảng PERSON để kế thừa thông tin cá nhân chung và là bảng cha của các loại khách hàng khác.                                                             |
| REGISTERED_CUSTOMER | Lưu thông tin khách hàng đã đăng ký tài khoản, bao gồm tên đăng nhập, email, mật khẩu, ngày đăng ký, điểm hiện tại và trạng thái tài khoản. Bảng này phục vụ các chức năng đăng nhập, tích điểm và quản lý tài khoản khách hàng. |
| GUEST_CUSTOMER      | Lưu thông tin khách hàng vắng lai, tức là khách hàng không có tài khoản đăng ký trong hệ thống. Bảng này giúp hệ thống vẫn có thể ghi nhận đơn hàng của khách chưa đăng ký tài khoản.                                            |
| ORDER               | Lưu thông tin đơn hàng của khách hàng, bao gồm trạng thái đơn hàng, ngày lập đơn, ghi chú, tổng tiền và khách hàng thực hiện đơn hàng. Đây là bảng trung tâm của phân hệ Khách hàng.                                             |
| ONLINE_ORDER        | Lưu thông tin các đơn hàng được đặt trực tuyến, bao gồm thời điểm phát hành vé, nền tảng đặt vé và phương thức nhận vé.                                                                                                          |
| IN_STORE_ORDER      | Lưu thông tin các đơn hàng được tạo trực tiếp tại quầy. Bảng này dùng để phân biệt với đơn hàng trực tuyến.                                                                                                                      |
| ORDER_DETAIL        | Lưu chi tiết từng dòng trong đơn hàng, bao gồm số lượng, đơn giá và thành tiền. Một đơn hàng có thể bao gồm nhiều chi tiết như vé xem phim hoặc sản phẩm đi kèm.                                                                 |
| TICKET_DETAIL       | Lưu thông tin chi tiết vé trong đơn hàng, bao gồm loại vé, giá vé và suất chiếu tương ứng. Bảng này liên kết đơn hàng với lịch chiếu phim.                                                                                       |
| PRODUCT_DETAIL      | Lưu thông tin chi tiết sản phẩm được mua trong đơn hàng, ví dụ như bắp nước hoặc các sản phẩm dịch vụ khác.                                                                                                                      |
| E_TICKET            | Lưu thông tin vé điện tử, bao gồm mã vé, mã QR, trạng thái vé, thời gian check-in và vị trí ghế. Bảng này giúp kiểm soát việc sử dụng vé và tránh trùng ghế.                                                                     |
| PAYMENT_TRANSACTION | Lưu thông tin giao dịch thanh toán của đơn hàng, bao gồm số tiền thanh toán, thời gian thanh toán, trạng thái, phương thức thanh toán và mã tham chiếu giao dịch.                                                                |

**Bảng 2.2:** Các bảng chính trong phân hệ phía Khách hàng

### 2.2.3 Các bảng hỗ trợ nghiệp vụ khách hàng

Ngoài các bảng chính, phân hệ Khách hàng còn có một số bảng hỗ trợ các nghiệp vụ mở rộng như hoàn đổi vé, in vé, đánh giá dịch vụ, tích điểm, sử dụng voucher và gửi yêu cầu phục vụ tại ghế.

| Bảng                    | Vai trò trong phân hệ phía Khách hàng                                                                                                                      |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TICKET_PRINT_INFO       | Lưu thông tin in vé của đơn hàng, bao gồm số lần in, thời gian in, lý do in, trạng thái in và nhân viên thực hiện. Bảng này giúp kiểm soát việc in lại vé. |
| REFUND_EXCHANGE_REQUEST | Lưu thông tin yêu cầu hoàn tiền hoặc đổi vé của khách hàng, bao gồm loại yêu cầu, lý do, thời gian xử lý, trạng thái và số tiền hoàn thực tế.              |
| PRODUCT                 | Lưu danh mục sản phẩm bán kèm tại rạp như bắp, nước uống hoặc combo. Bảng này được liên kết với chi tiết sản phẩm trong đơn hàng.                          |
| REVIEW                  | Lưu thông tin đánh giá của khách hàng, bao gồm nội dung đánh giá, thời gian đánh giá và số sao.                                                            |
| PRODUCT_REVIEW          | Lưu mối liên hệ giữa đánh giá và sản phẩm. Bảng này cho phép khách hàng đánh giá các sản phẩm đã sử dụng.                                                  |
| ORDER_REVIEW            | Lưu mối liên hệ giữa đánh giá và đơn hàng. Bảng này cho phép khách hàng đánh giá trải nghiệm mua vé hoặc sử dụng dịch vụ.                                  |
| POINT_HISTORY           | Lưu lịch sử thay đổi điểm tích lũy của khách hàng, bao gồm số điểm thay đổi, lý do thay đổi, thời gian phát sinh và đơn hàng liên quan.                    |
| VOUCHER                 | Lưu thông tin mã giảm giá, bao gồm loại giảm giá, giá trị giảm, điều kiện áp dụng, thời hạn sử dụng và trạng thái voucher.                                 |
| APPLY                   | Lưu thông tin áp dụng voucher vào đơn hàng, bao gồm mã voucher, mã đơn hàng và số tiền được giảm.                                                          |
| SERVICE_REQUEST         | Lưu yêu cầu dịch vụ tại ghế của khách hàng, bao gồm vị trí ghế, đơn hàng, nội dung yêu cầu, trạng thái xử lý và thời gian yêu cầu.                         |

**Bảng 2.3:** Các bảng hỗ trợ nghiệp vụ trong phân hệ phía Khách hàng

#### 2.2.4 Mối quan hệ giữa các bảng

Các bảng trong phân hệ Khách hàng được thiết kế theo mô hình quan hệ nhằm đảm bảo dữ liệu được tổ chức chặt chẽ và hạn chế trùng lặp. Bảng **CUSTOMER** đóng vai trò là bảng trung tâm để quản lý khách hàng và được liên kết với bảng **PERSON**. Từ bảng **CUSTOMER**, hệ thống phân loại khách hàng thành khách hàng có tài khoản trong bảng **REGISTERED\_CUSTOMER** và khách hàng vắng lai trong bảng **GUEST\_CUSTOMER**.

Bảng **ORDER** là bảng trung tâm của nghiệp vụ giao dịch. Mỗi đơn hàng được gắn với một khách hàng cụ thể thông qua **PersonID**. Tùy theo hình thức mua hàng, đơn hàng có thể là đơn hàng trực tuyến trong bảng **ONLINE\_ORDER** hoặc đơn hàng tại quầy trong bảng **IN\_STORE\_ORDER**.

Mỗi đơn hàng có thể bao gồm nhiều dòng chi tiết trong bảng **ORDER\_DETAIL**. Từ bảng này, hệ thống tiếp tục phân loại chi tiết đơn hàng thành chi tiết vé trong bảng **TICKET\_DETAIL** hoặc chi tiết sản phẩm trong bảng **PRODUCT\_DETAIL**. Đối với vé xem phim, bảng **TICKET\_DETAIL** liên kết với bảng **SHOWTIME** để xác định suất chiếu, đồng thời bảng **E\_TICKET** lưu thông tin vé điện tử và vị trí ghế cụ thể của khách hàng.

Bên cạnh đó, bảng **PAYMENT\_TRANSACTION** liên kết với bảng **ORDER** để ghi nhận các giao dịch thanh toán. Các bảng **VOUCHER** và **APPLY** được sử dụng để quản lý việc áp dụng mã giảm giá vào đơn hàng. Bảng **POINT\_HISTORY** lưu lại lịch sử thay đổi điểm tích lũy của khách hàng sau mỗi giao dịch hoặc hoạt động liên quan.

Ngoài ra, hệ thống còn hỗ trợ các nghiệp vụ sau bán hàng thông qua các bảng `REFUND_EXCHANGE_REQUEST`, `REVIEW`, `ORDER_REVIEW`, `PRODUCT_REVIEW` và `SERVICE_REQUEST`. Những bảng này giúp hệ thống ghi nhận phản hồi, xử lý yêu cầu hoàn đổi và hỗ trợ khách hàng trong quá trình sử dụng dịch vụ tại rạp.

## 2.2.5 Ràng buộc dữ liệu chính

Trong quá trình hiện thực cơ sở dữ liệu, các ràng buộc dữ liệu đơn giản được cài đặt trực tiếp trong câu lệnh tạo bảng bằng khóa chính, khóa ngoại, `NOT NULL`, `UNIQUE`, `CHECK` và giá trị mặc định. Việc cài đặt ràng buộc ngay trong bảng giúp hệ thống kiểm soát dữ liệu đầu vào, đảm bảo tính chính xác và hạn chế lỗi phát sinh trong quá trình vận hành.

Đối với phân hệ phía Khách hàng, các ràng buộc chính gồm:

- Các bảng như `CUSTOMER`, `ORDER`, `ORDER_DETAIL`, `PRODUCT`, `E_TICKET`, `PAYMENT_TRANSACTION`, `REVIEW`, `VOUCHER` và `SERVICE_REQUEST` đều có khóa chính để định danh từng dòng dữ liệu.
- Các quan hệ giữa bảng được kiểm soát bằng khóa ngoại, chẳng hạn `ORDER` gắn với `CUSTOMER`, `ORDER_DETAIL` gắn với `ORDER`, `TICKET_DETAIL` gắn với `SHOWTIME`, `PRODUCT_DETAIL` gắn với `PRODUCT`, `E_TICKET` gắn với `SEAT`, và `PAYMENT_TRANSACTION` gắn với `ORDER`.
- Các thông tin tài khoản khách hàng đăng ký như `Username` và `Email` được đặt ràng buộc `UNIQUE` nhằm tránh trùng lặp tài khoản trong hệ thống.
- Các thuộc tính số như `CurrentPoints`, `TotalAmount`, `OrderDetailQuantity`, `OrderDetailUnitPrice`, `OrderDetailSubtotal`, `BasePrice`, `TicketDetailTicketPrice`, `PaymentTransactionAmount`, `VoucherValue` và `DiscountAmount` được kiểm tra để tránh nhận giá trị âm hoặc không hợp lệ.
- Số sao đánh giá trong bảng `REVIEW` được giới hạn từ 1 đến 5 nhằm đảm bảo phù hợp với thang đánh giá thông thường.
- Loại giảm giá của voucher trong bảng `VOUCHER` chỉ nhận một trong hai giá trị `Percent` hoặc `FixedAmount`.
- Trạng thái voucher được giới hạn trong các giá trị hợp lệ như `Active`, `Expired` và `Disabled`.
- Trạng thái yêu cầu dịch vụ tại ghế trong bảng `SERVICE_REQUEST` được giới hạn trong các giá trị `Pending`, `InProgress`, `Completed` và `Canceled`.
- Vị trí ghế trong bảng `E_TICKET` và `SERVICE_REQUEST` được liên kết với bảng `SEAT` nhằm đảm bảo ghế được chọn là ghế tồn tại trong phòng chiếu.

Một số ràng buộc nghiệp vụ phức tạp hơn không thể kiểm tra đầy đủ bằng `CHECK` hoặc khóa ngoại sẽ được xử lý bằng trigger ở chương sau. Ví dụ, hệ thống cần kiểm tra việc đặt vé không được trùng ghế trong cùng một suất chiếu, voucher chỉ được áp dụng khi còn hiệu lực, hoặc tổng tiền đơn hàng cần được cập nhật tự động dựa trên các dòng chi tiết đơn hàng.

## 2.2.6 Dữ liệu mẫu

Dữ liệu mẫu được tạo cho các bảng trong phân hệ phía Khách hàng nhằm phục vụ quá trình kiểm thử, demo và đánh giá hoạt động của hệ thống. Dữ liệu bao gồm thông tin khách hàng đăng ký, khách hàng vắng lai, đơn hàng trực tuyến, đơn hàng tại quầy, chi tiết đơn hàng, vé điện tử, sản phẩm, giao dịch

thanh toán, voucher, lịch sử điểm, đánh giá và yêu cầu dịch vụ tại ghế.

Dữ liệu mẫu được xây dựng sao cho phản ánh được các tình huống nghiệp vụ phổ biến trong thực tế. Ví dụ, một khách hàng có thể tạo đơn hàng mua vé xem phim, áp dụng voucher, thanh toán thành công, nhận vé điện tử, tích điểm sau giao dịch và gửi đánh giá sau khi sử dụng dịch vụ. Ngoài ra, hệ thống cũng có thể có các trường hợp đơn hàng bị hủy, yêu cầu hoàn tiền, in lại vé hoặc gửi yêu cầu hỗ trợ tại ghế.

Các dữ liệu này được dùng để kiểm tra hoạt động của khóa chính, khóa ngoại, ràng buộc CHECK, các thủ tục thêm, sửa, xóa dữ liệu, các trigger nghiệp vụ và các hàm tính toán trong hệ thống. Toàn bộ script tạo bảng, tạo ràng buộc và thêm dữ liệu mẫu được nộp kèm trong file SQL của nhóm.

## 2.3 Nhận xét

Cách thiết kế này giúp cơ sở dữ liệu có khả năng mở rộng khi cần bổ sung các chức năng mới như đặt vé trực tuyến, tích hợp thanh toán online, quản lý hạng thành viên hoặc chăm sóc khách hàng. Đồng thời, việc sử dụng các ràng buộc dữ liệu ngay trong cơ sở dữ liệu giúp đảm bảo tính toàn vẹn và giảm thiểu sai sót trong quá trình nhập liệu, cập nhật và xử lý giao dịch.

# CHƯƠNG 3. THỦ TỤC THÊM, SỬA, XÓA DỮ LIỆU

## 3.1 Bảng EVENT

### 3.1.1 Lý do chọn bảng EVENT

Trong phần hiện thực thủ tục thêm, sửa và xóa dữ liệu, nhóm chọn bảng **EVENT** làm bảng chính. Đây là bảng lưu thông tin các sự kiện được tổ chức tại rạp, bao gồm nhà tổ chức, tên sự kiện, thời gian diễn ra, quy mô dự kiến, ngân sách và trạng thái sự kiện.

Bảng **EVENT** phù hợp với phần phía Rạp vì dữ liệu sự kiện có nhiều ràng buộc cần kiểm tra. Một sự kiện không chỉ cần có thông tin cơ bản hợp lệ, mà còn phải liên kết đúng với nhà tổ chức, hợp đồng và các phiên tổ chức trong bảng **SESSION**. Do đó, các thao tác thêm, sửa và xóa trên bảng này được hiện thực bằng thủ tục riêng thay vì chỉ dùng trực tiếp các câu lệnh **INSERT**, **UPDATE** hoặc **DELETE**.

Ba thủ tục được hiện thực gồm:

- **usp\_Event\_Insert**: thêm sự kiện mới.
- **usp\_Event\_Update**: cập nhật thông tin sự kiện.
- **usp\_Event\_Delete**: xóa sự kiện khi thỏa điều kiện nghiệp vụ.

### 3.1.2 Thủ tục thêm sự kiện

Thủ tục **usp\_Event\_Insert** được dùng để thêm một sự kiện mới vào bảng **EVENT**. Trước khi thêm dữ liệu, thủ tục kiểm tra các thông tin đầu vào để tránh tạo ra sự kiện không hợp lệ.

Các điều kiện chính được kiểm tra gồm:

- Mã sự kiện không được để trống và không được trùng với sự kiện đã tồn tại.
- Mã nhà tổ chức không được để trống và phải tồn tại trong bảng **ORGANIZER**.
- Tên sự kiện không được để trống.
- Ngày bắt đầu và ngày kết thúc phải hợp lệ.
- Ngày kết thúc không được nhỏ hơn ngày bắt đầu.
- Quy mô dự kiến phải lớn hơn 0.
- Tổng ngân sách không được âm.
- Trạng thái sự kiện phải thuộc các giá trị hợp lệ: **Chờ duyệt**, **Đã duyệt**, **Từ chối**, **Đã hoàn thành**.

Ngoài các kiểm tra cơ bản, thủ tục còn xử lý trường hợp nhà tổ chức thuộc loại **Corporate**. Nếu sự kiện của nhà tổ chức **Corporate** được tạo ở trạng thái **Đã duyệt**, thời gian diễn ra sự kiện phải nằm trong thời hạn hiệu lực của một hợp đồng tương ứng trong bảng **CONTRACT**. Điều kiện này giúp đảm bảo sự kiện đã được duyệt có cơ sở hợp đồng hợp lệ.

```

1 IF @OrganizerType = N'Corporate'
2   AND @EventStatus = N'Da duyet'
3   AND NOT EXISTS (
4     SELECT 1
5     FROM CONTRACT
6     WHERE CONTRACT.OrganizerID = @OrganizerID
7           AND @EventStartDate >= EffectiveDate
8           AND @EventEndDate <= TerminationDate
9   )
10 BEGIN
11   RAISERROR(N'Nhà tổ chức Corporate chỉ được tạo sự kiện Đã duyệt khi thời gian sự kiện
12     nằm trong hiệu lực hợp đồng.', 16, 1);
13   RETURN;
14 END;

```

Listing 3.1: Kiểm tra thời hạn hợp đồng khi thêm sự kiện

Sau khi các điều kiện đều hợp lệ, thủ tục thực hiện thêm dữ liệu vào bảng **EVENT** và trả về thông tin sự kiện vừa được thêm.

### 3.1.3 Thủ tục sửa sự kiện

Thủ tục **usp\_Event\_Update** được dùng để cập nhật thông tin của một sự kiện đã tồn tại. Tương tự thủ tục thêm, thủ tục sửa kiểm tra các dữ liệu đầu vào như mã sự kiện, mã nhà tổ chức, tên sự kiện, thời gian diễn ra, quy mô dự kiến, ngân sách và trạng thái sự kiện.

Điểm khác biệt của thủ tục sửa là phải xét thêm dữ liệu đã phát sinh liên quan đến sự kiện. Trước hết, thủ tục kiểm tra sự kiện cần sửa có tồn tại trong bảng **EVENT** hay không. Nếu không tìm thấy mã sự kiện, thủ tục trả về lỗi và không thực hiện cập nhật.

Một số điều kiện nghiệp vụ quan trọng trong thủ tục sửa gồm:

- Không được đổi nhà tổ chức nếu sự kiện đã có phiên tổ chức trong bảng **SESSION**.
- Khoảng thời gian mới của sự kiện phải bao phủ toàn bộ các phiên **SESSION** hiện có.
- Không được chuyển sự kiện sang trạng thái **Từ chối** nếu đã có phiên ở trạng thái **Đã xác nhận**.
- Nếu nhà tổ chức là **Corporate** và sự kiện được cập nhật sang trạng thái **Đã duyệt**, thời gian sự kiện phải nằm trong hiệu lực hợp đồng.

Các kiểm tra này nhằm tránh việc cập nhật sự kiện làm dữ liệu đã phát sinh trở nên không nhất quán. Ví dụ, nếu một sự kiện đã có phiên tổ chức thì việc đổi nhà tổ chức có thể làm sai lệch thông tin liên quan đến hợp đồng và lịch tổ chức. Tương tự, nếu thu hẹp thời gian sự kiện khiến một số phiên tổ chức nằm ngoài khoảng thời gian mới thì dữ liệu giữa **EVENT** và **SESSION** sẽ không còn phù hợp.

```

1 IF EXISTS (
2   SELECT 1
3   FROM SESSION
4   WHERE EventID = @EventID
5         AND (
6           CAST(SessionStart AS DATE) < @EventStartDate
7           OR CAST(SessionEnd AS DATE) > @EventEndDate
8         )
9 )
10 BEGIN

```

```
11 RAISERROR(N'Khoang thoi gian moi cua EVENT khong bao phu toan bo SESSION dang co.',  
12 16, 1);  
12 RETURN;  
13 END;
```

Listing 3.2: Kiểm tra SESSION khi cập nhật thời gian sự kiện

Sau khi vượt qua các bước kiểm tra, thủ tục cập nhật các thuộc tính của sự kiện trong bảng **EVENT** và trả về dữ liệu sau khi cập nhật.

### 3.1.4 Thủ tục xóa sự kiện

Thủ tục **usp\_Event\_Delete** được dùng để xóa một sự kiện khỏi bảng **EVENT**. Tuy nhiên, không phải mọi sự kiện đều được phép xóa. Trong hệ thống quản lý rạp, sự kiện có thể liên quan đến lịch tổ chức, phòng chiếu và các phiên trong bảng **SESSION**. Vì vậy, thao tác xóa cần được kiểm soát để tránh làm mất dữ liệu nghiệp vụ đã phát sinh.

Thủ tục chỉ cho phép xóa sự kiện khi thỏa các điều kiện sau:

- Mã sự kiện được truyền vào không được để trống.
- Sự kiện cần xóa phải tồn tại trong bảng **EVENT**.
- Sự kiện chưa phát sinh phiên tổ chức nào trong bảng **SESSION**.
- Trạng thái sự kiện phải là **Chờ duyệt** hoặc **Từ chối**.

Nếu sự kiện đã có dữ liệu trong bảng **SESSION**, thủ tục không cho phép xóa. Lý do là các phiên tổ chức đã phát sinh có thể liên quan đến lịch sử sử dụng phòng, phí thuê và trạng thái xác nhận. Trong trường hợp này, việc xóa trực tiếp sự kiện có thể làm mất thông tin cần thiết cho các chức năng kiểm tra và báo cáo sau này.

```
1 IF EXISTS (SELECT 1 FROM SESSION WHERE EventID = @EventID)  
2 BEGIN  
3     RAISERROR(N'Khong duoc xoa su kien vi da phat sinh SESSION.', 16, 1);  
4     RETURN;  
5 END;  
6  
7 IF @EventStatus NOT IN (N'Cho duyet', N'Tu choi')  
8 BEGIN  
9     RAISERROR(N'Chi duoc xoa su kien o trang thai Cho duyet hoac Tu choi.', 16, 1);  
10    RETURN;  
11 END;
```

Listing 3.3: Kiểm tra điều kiện xóa sự kiện

Như vậy, thủ tục xóa chủ yếu được dùng cho các sự kiện nhập sai, chưa được triển khai hoặc đã bị từ chối. Đối với các sự kiện đã được duyệt, đã hoàn thành hoặc đã có phiên tổ chức, hệ thống không xóa trực tiếp để bảo toàn dữ liệu nghiệp vụ.

### 3.1.5 Nhận xét

Các thủ tục thêm, sửa và xóa trên bảng **EVENT** không chỉ thực hiện thao tác dữ liệu cơ bản mà còn kiểm tra các ràng buộc nghiệp vụ liên quan. Mỗi lỗi dữ liệu đều được trả về bằng thông báo cụ thể, giúp người dùng hoặc tăng ứng dụng biết rõ nguyên nhân thao tác không thành công.

Cách hiện thực này giúp hạn chế dữ liệu không hợp lệ đi vào hệ thống, đồng thời đảm bảo các

thao tác trên sự kiện không làm ảnh hưởng đến dữ liệu đã phát sinh trong bảng **SESSION** và các bảng liên quan.

## 3.2 Bảng **PRODUCT**

### 3.2.1 Lý do chọn bảng **PRODUCT**

Trong phần hiện thực thủ tục thêm, sửa và xóa dữ liệu ở phía Khách hàng, nhóm chọn bảng **PRODUCT** làm bảng chính. Đây là bảng lưu thông tin các sản phẩm được bán kèm tại rạp chiếu phim như bắp, nước, combo hoặc các sản phẩm khác. Bảng **PRODUCT** bao gồm các thông tin quan trọng như mã sản phẩm, tên sản phẩm, loại sản phẩm, giá cơ bản và trạng thái kinh doanh của sản phẩm.

Bảng **PRODUCT** phù hợp với phân hệ phía Khách hàng vì sản phẩm là một phần quan trọng trong quá trình khách hàng tạo đơn hàng. Khi khách hàng mua vé xem phim, ngoài vé, khách hàng có thể mua thêm các sản phẩm đi kèm. Các sản phẩm này được liên kết với chi tiết đơn hàng thông qua bảng **PRODUCT\_DETAIL**. Do đó, việc thêm, sửa và xóa sản phẩm cần được kiểm soát chặt chẽ để tránh ảnh hưởng đến dữ liệu đơn hàng đã phát sinh.

Ba thủ tục được hiện thực gồm:

- **sp\_InsertProduct**: thêm sản phẩm mới.
- **sp\_UpdateProduct**: cập nhật thông tin sản phẩm.
- **sp\_DeleteProduct**: xóa sản phẩm khi thỏa điều kiện nghiệp vụ.

### 3.2.2 Thủ tục thêm sản phẩm

Thủ tục **sp\_InsertProduct** được dùng để thêm một sản phẩm mới vào bảng **PRODUCT**. Trước khi thêm dữ liệu, thủ tục thực hiện chuẩn hóa dữ liệu đầu vào bằng cách loại bỏ khoảng trắng thừa ở đầu và cuối các chuỗi nhập vào. Việc này giúp hạn chế lỗi nhập liệu do người dùng vô tình nhập dư khoảng trắng.

Các điều kiện chính được kiểm tra gồm:

- Mã sản phẩm không được để trống.
- Mã sản phẩm không được trùng với sản phẩm đã tồn tại trong bảng **PRODUCT**.
- Tên sản phẩm không được để trống.
- Loại sản phẩm không được để trống.
- Loại sản phẩm phải thuộc một trong các giá trị hợp lệ: **Bắp, Nước, Combo, Khác**.
- Giá cơ bản không được để trống.
- Giá cơ bản phải lớn hơn hoặc bằng 0.
- Trạng thái sản phẩm không được để trống.
- Trạng thái sản phẩm phải thuộc một trong các giá trị hợp lệ: **Đang bán, Tạm ngừng, Ngừng kinh doanh**.

Nếu dữ liệu đầu vào không hợp lệ, thủ tục sử dụng **THROW** để trả về thông báo lỗi cụ thể. Ví dụ,

nếu mã sản phẩm bị bỏ trống, hệ thống sẽ báo lỗi mã sản phẩm không được để trống; nếu loại sản phẩm không thuộc danh sách cho phép, hệ thống sẽ báo loại sản phẩm không hợp lệ.

```
1 IF @ProductID IS NULL OR @ProductID = ''
2     THROW 50001, N'Mã sản phẩm không được để trống.', 1;
3
4 IF EXISTS (
5     SELECT 1
6     FROM PRODUCT
7     WHERE ProductID = @ProductID
8 )
9     THROW 50009, N'Mã sản phẩm đã tồn tại.', 1;
10
11 IF @ProductType NOT IN (N'Bếp', N'Nước', N'Combo', N'Khác')
12     THROW 50004, N'Loại sản phẩm không hợp lệ. Chỉ chấp nhận: Bếp, Nước, Combo, Khác.',
13     1;
14
15 IF @BasePrice < 0
16     THROW 50006, N'Giá cơ bản phải lớn hơn hoặc bằng 0.', 1;
```

Listing 3.4: Kiểm tra dữ liệu khi thêm sản phẩm

Sau khi tất cả dữ liệu đều hợp lệ, thủ tục thực hiện câu lệnh `INSERT` để thêm sản phẩm mới vào bảng `PRODUCT`. Việc kiểm tra dữ liệu trước khi thêm giúp đảm bảo rằng bảng sản phẩm không chứa các bản ghi thiếu thông tin, sai loại sản phẩm hoặc có giá trị giá bán không hợp lệ.

### 3.2.3 Thủ tục sửa sản phẩm

Thủ tục `sp_UpdateProduct` được dùng để cập nhật thông tin của một sản phẩm đã tồn tại trong bảng `PRODUCT`. Tương tự thủ tục thêm, thủ tục sửa cũng thực hiện chuẩn hóa dữ liệu đầu vào và kiểm tra tính hợp lệ của các thông tin được truyền vào.

Trước hết, thủ tục kiểm tra mã sản phẩm có được nhập hay không. Sau đó, hệ thống kiểm tra sản phẩm cần cập nhật có tồn tại trong bảng `PRODUCT` hay không. Nếu không tìm thấy sản phẩm tương ứng, thủ tục dừng xử lý và trả về thông báo lỗi.

Các điều kiện nghiệp vụ chính trong thủ tục sửa gồm:

- Mã sản phẩm không được để trống.
- Sản phẩm cần cập nhật phải tồn tại trong bảng `PRODUCT`.
- Tên sản phẩm không được để trống.
- Loại sản phẩm phải thuộc các giá trị hợp lệ.
- Giá cơ bản phải lớn hơn hoặc bằng 0.
- Trạng thái sản phẩm phải thuộc các giá trị hợp lệ.

```
1 IF @ProductID IS NULL OR @ProductID = ''
2     THROW 50101, N'Mã sản phẩm không được để trống.', 1;
3
4 IF NOT EXISTS (
5     SELECT 1
6     FROM PRODUCT
7     WHERE ProductID = @ProductID
```

```

8 )
9     THROW 50102, N'Không tìm thấy sản phẩm cần cập nhật.', 1;

```

Listing 3.5: Kiểm tra sản phẩm tồn tại trước khi cập nhật  
Sau khi dữ liệu đầu vào hợp lệ, thủ tục thực hiện cập nhật các thông tin gồm tên sản phẩm, loại sản phẩm, giá cơ bản và trạng thái sản phẩm.

```

1 UPDATE PRODUCT
2 SET
3     ProductName = @ProductName,
4     ProductType = @ProductType,
5     BasePrice = @BasePrice,
6     ProductStatus = @ProductStatus
7 WHERE ProductID = @ProductID;

```

Listing 3.6: Cập nhật thông tin sản phẩm  
Việc hiện thực thủ tục sửa giúp đảm bảo rằng khi thông tin sản phẩm thay đổi, dữ liệu vẫn tuân thủ đúng các ràng buộc nghiệp vụ. Ví dụ, người dùng không thể cập nhật loại sản phẩm thành một giá trị không xác định hoặc cập nhật giá sản phẩm thành số âm.

### 3.2.4 Thủ tục xóa sản phẩm

Thủ tục `sp_DeleteProduct` được dùng để xóa một sản phẩm khỏi bảng `PRODUCT`. Tuy nhiên, không phải mọi sản phẩm đều được phép xóa trực tiếp. Trong hệ thống rạp chiếu phim, sản phẩm có thể đã từng được khách hàng mua và phát sinh trong các đơn hàng. Nếu xóa sản phẩm không kiểm soát, dữ liệu trong bảng `PRODUCT_DETAIL`, `ORDER_DETAIL` và `ORDER` có thể bị ảnh hưởng.

Vì vậy, thủ tục xóa sản phẩm được xây dựng với các điều kiện kiểm tra nghiệp vụ trước khi thực hiện xóa. Các điều kiện chính gồm:

- Mã sản phẩm không được để trống.
- Sản phẩm cần xóa phải tồn tại trong bảng `PRODUCT`.
- Chỉ được xóa sản phẩm có trạng thái `Ngừng kinh doanh`.
- Không được xóa sản phẩm nếu trong ngày hiện tại đã phát sinh đơn hàng chứa sản phẩm đó.

Điều kiện sản phẩm phải có trạng thái `Ngừng kinh doanh` trước khi xóa giúp đảm bảo rằng sản phẩm không còn được sử dụng trong hoạt động bán hàng. Điều này phù hợp với nghiệp vụ thực tế, vì một sản phẩm đang bán hoặc tạm ngừng bán không nên bị xóa ngay khỏi hệ thống.

```

1 IF NOT EXISTS (
2     SELECT 1
3     FROM PRODUCT
4     WHERE ProductID = @ProductID
5           AND ProductStatus = N'Ngừng kinh doanh'
6 )
7     THROW 50203, N'Chỉ được xóa sản phẩm có trạng thái "Ngừng kinh doanh". Hãy cập nhật trạng thái trước khi xóa.', 1;

```

Listing 3.7: Kiểm tra trạng thái sản phẩm trước khi xóa  
Ngoài ra, thủ tục còn kiểm tra xem trong ngày hiện tại có đơn hàng nào chứa sản phẩm cần xóa hay không. Việc kiểm tra này được thực hiện bằng cách liên kết bảng `PRODUCT_DETAIL`, `ORDER_DETAIL`

và ORDER. Nếu sản phẩm đã phát sinh trong đơn hàng của ngày hiện tại, thủ tục không cho phép xóa để tránh ảnh hưởng đến dữ liệu giao dịch mới phát sinh.

```
1 IF EXISTS (  
2     SELECT 1  
3     FROM PRODUCT_DETAIL pd  
4     JOIN ORDER_DETAIL od  
5         ON pd.OrderDetailID = od.OrderDetailID  
6     JOIN [ORDER] o  
7         ON od.OrderID = o.OrderID  
8     WHERE pd.ProductID = @ProductID  
9         AND o.OrderDate = CAST(GETDATE() AS DATE)  
10 )  
11     THROW 50204, N'Không thể xóa vì hôm nay đã phát sinh đơn hàng chứa sản phẩm này.', 1;
```

Listing 3.8: Kiểm tra đơn hàng phát sinh trước khi xóa sản phẩm

Nếu sản phẩm thỏa tất cả điều kiện xóa, thủ tục thực hiện câu lệnh DELETE để xóa sản phẩm khỏi bảng PRODUCT.

```
1 DELETE FROM PRODUCT  
2 WHERE ProductID = @ProductID;
```

Listing 3.9: Xóa sản phẩm khỏi bảng PRODUCT

Như vậy, thủ tục xóa sản phẩm chủ yếu được dùng cho các sản phẩm đã ngừng kinh doanh và không còn phát sinh giao dịch trong ngày. Đối với các sản phẩm đang bán, tạm ngừng bán hoặc đã có đơn hàng mới phát sinh, hệ thống không xóa trực tiếp nhằm bảo toàn dữ liệu nghiệp vụ.

### 3.2.5 Nhận xét

Các thủ tục thêm, sửa và xóa trên bảng PRODUCT không chỉ thực hiện các thao tác dữ liệu cơ bản mà còn kiểm tra các ràng buộc nghiệp vụ cần thiết. Mỗi lỗi dữ liệu đều được trả về bằng thông báo cụ thể thông qua THROW, giúp người dùng hoặc tầng ứng dụng xác định rõ nguyên nhân thao tác thất bại.

Việc lựa chọn bảng PRODUCT để hiện thực thủ tục thêm, sửa và xóa là phù hợp với phân hệ phía Khách hàng. Bảng này có liên quan trực tiếp đến quá trình mua hàng, chi tiết đơn hàng và doanh thu sản phẩm. Cách hiện thực này giúp hạn chế dữ liệu không hợp lệ, tránh xóa nhầm dữ liệu đang được sử dụng và đảm bảo tính nhất quán giữa bảng PRODUCT với các bảng liên quan như PRODUCT\_DETAIL, ORDER\_DETAIL và ORDER.

## CHƯƠNG 4. TRIGGER

### 4.1 Trigger trên bảng SESSION

Trong phân hệ phía Rạp, bảng **SESSION** được dùng để lưu các phiên tổ chức cụ thể của từng sự kiện. Mỗi phiên sự kiện gắn với một sự kiện trong bảng **EVENT** và được tổ chức tại một phòng chiếu cụ thể. Do đó, khi thêm, sửa hoặc xóa dữ liệu trong bảng **SESSION**, hệ thống cần kiểm tra một số điều kiện nghiệp vụ để đảm bảo dữ liệu giữa **EVENT** và **SESSION** không bị sai lệch.

Trong phần này, nhóm hiện thực hai trigger chính:

- **TRG\_Session\_Inside\_Event\_Date**: kiểm tra phiên sự kiện phải nằm trong khoảng thời gian diễn ra sự kiện.
- **TRG\_Session\_Update\_ConfirmedSessionCount**: cập nhật thuộc tính dẫn xuất **ConfirmedSessionCount** của bảng **EVENT**.

#### 4.1.1 Trigger kiểm tra thời gian phiên sự kiện

Trigger **TRG\_Session\_Inside\_Event\_Date** được dùng để kiểm tra ràng buộc nghiệp vụ giữa bảng **SESSION** và bảng **EVENT**. Ràng buộc được chọn là: mỗi phiên sự kiện phải nằm trong khoảng thời gian bắt đầu và kết thúc của sự kiện tương ứng.

Cụ thể, nếu một sự kiện có ngày bắt đầu là **EventStartDate** và ngày kết thúc là **EventEndDate**, thì các giá trị **SessionStart** và **SessionEnd** của phiên sự kiện không được nằm ngoài khoảng ngày này. Ràng buộc này không thể kiểm tra đầy đủ bằng **CHECK** trong câu lệnh tạo bảng vì cần so sánh dữ liệu giữa hai bảng khác nhau. Vì vậy, nhóm sử dụng trigger để kiểm tra sau thao tác thêm hoặc cập nhật trên bảng **SESSION**.

Các thao tác có thể làm vi phạm ràng buộc gồm:

- **INSERT** một phiên sự kiện mới có thời gian nằm ngoài thời gian của sự kiện.
- **UPDATE** thời gian hoặc mã sự kiện của một phiên, làm cho phiên đó không còn nằm trong khoảng thời gian hợp lệ.

```
1 CREATE TRIGGER TRG_Session_Inside_Event_Date
2 ON SESSION
3 AFTER INSERT, UPDATE
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7
8     IF EXISTS (
9         SELECT 1
10        FROM inserted
11        JOIN EVENT ON inserted.EventID = EVENT.EventID
12        WHERE CAST(inserted.SessionStart AS DATE) < EVENT.EventStartDate
13              OR CAST(inserted.SessionEnd AS DATE) > EVENT.EventEndDate
14    )
15 BEGIN
```

```
16 RAISERROR(N'SESSION phải nằm trong khoảng ngày của EVENT.', 16, 1);
17 ROLLBACK TRANSACTION;
18 RETURN;
19 END;
20 END;
```

Listing 4.1: Trigger kiểm tra phiên sự kiện nằm trong thời gian của EVENT

Trong trigger này, bảng tạm `inserted` được sử dụng để lấy các dòng vừa được thêm hoặc cập nhật. Nếu tồn tại ít nhất một dòng có thời gian phiên nằm ngoài thời gian của sự kiện cha, trigger sẽ phát sinh lỗi và rollback giao dịch hiện tại. Nhờ đó, dữ liệu không hợp lệ sẽ không được ghi vào cơ sở dữ liệu.

#### 4.1.2 Trigger cập nhật thuộc tính dẫn xuất

Thuộc tính dẫn xuất được chọn là `ConfirmedSessionCount` trong bảng `EVENT`. Thuộc tính này lưu số lượng phiên sự kiện đã được xác nhận của từng sự kiện. Giá trị này được tính dựa trên số dòng trong bảng `SESSION` có cùng `EventID` và có `SessionStatus = N'Đã xác nhận'`.

Các thao tác trên bảng `SESSION` có thể làm thay đổi giá trị `ConfirmedSessionCount` gồm:

- INSERT: thêm một phiên mới, đặc biệt nếu phiên đó có trạng thái `Đã xác nhận`.
- UPDATE: thay đổi trạng thái phiên, hoặc chuyển phiên từ sự kiện này sang sự kiện khác.
- DELETE: xóa một phiên đã tồn tại.

Do giá trị `ConfirmedSessionCount` phụ thuộc vào dữ liệu của bảng `SESSION`, nhóm sử dụng trigger để tự động cập nhật lại thuộc tính này sau mỗi thao tác thêm, sửa hoặc xóa trên bảng `SESSION`.

```
1 CREATE TRIGGER TRG_Session_Update_ConfirmedSessionCount
2 ON SESSION
3 AFTER INSERT, UPDATE, DELETE
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7
8     UPDATE EVENT
9     SET ConfirmedSessionCount = 0
10    WHERE EventID IN (
11        SELECT EventID FROM inserted WHERE EventID IS NOT NULL
12        UNION
13        SELECT EventID FROM deleted WHERE EventID IS NOT NULL
14    );
15
16    UPDATE EVENT
17    SET ConfirmedSessionCount = Temp.Cnt
18    FROM EVENT
19    JOIN (
20        SELECT EventID, COUNT(*) AS Cnt
21        FROM SESSION
22        WHERE SessionStatus = N'Đã xác nhận'
23        AND EventID IN (
24            SELECT EventID FROM inserted WHERE EventID IS NOT NULL
25            UNION
26            SELECT EventID FROM deleted WHERE EventID IS NOT NULL
27        )
28    GROUP BY EventID
```

```
29 ) AS Temp
30     ON EVENT.EventID = Temp.EventID;
31 END;
```

Listing 4.2: Trigger cập nhật số phiên đã xác nhận của EVENT.

Trigger này xử lý cả hai bảng tạm `inserted` và `deleted`. Cách làm này cần thiết vì thao tác UPDATE hoặc DELETE có thể ảnh hưởng đến các dòng cũ, trong khi thao tác INSERT tạo ra các dòng mới. Việc lấy EventID từ cả hai bảng tạm giúp xác định đúng những sự kiện cần được cập nhật lại số phiên đã xác nhận.

Trước hết, trigger đặt `ConfirmedSessionCount` của các sự kiện bị ảnh hưởng về 0. Sau đó, trigger tính lại số phiên có trạng thái **Đã xác nhận** trong bảng `SESSION` và cập nhật lại cho từng sự kiện tương ứng. Cách làm này giúp tránh trường hợp một sự kiện không còn phiên xác nhận nào nhưng vẫn giữ giá trị cũ.

### 4.1.3 Nhận xét

Hai trigger trên được đặt trên bảng `SESSION` vì đây là bảng trực tiếp làm thay đổi lịch tổ chức và số phiên đã xác nhận của sự kiện. Trigger `TRG_Session_Inside_Event_Date` dùng để kiểm tra ràng buộc nghiệp vụ giữa phiên sự kiện và sự kiện cha. Trigger `TRG_Session_Update_ConfirmedSessionCount` dùng để cập nhật thuộc tính dẫn xuất trong bảng `EVENT`.

Các trigger đều được viết theo hướng xử lý nhiều dòng dữ liệu, không chỉ xử lý một dòng đơn lẻ. Điều này phù hợp với cách hoạt động của SQL Server, vì một câu lệnh INSERT, UPDATE hoặc DELETE có thể tác động đến nhiều dòng cùng lúc.

## 4.2 Trigger phía Khách hàng

Trong phân hệ phía Khách hàng, các bảng như `ORDER`, `ORDER_DETAIL`, `APPLY`, `POINT_HISTORY`, `REGISTERED_CUSTOMER`, `REVIEW`, `ORDER_REVIEW` và `PRODUCT_REVIEW` được sử dụng để quản lý quá trình khách hàng mua vé, mua sản phẩm, thanh toán, sử dụng voucher, tích điểm và đánh giá dịch vụ.

Các nghiệp vụ này có nhiều ràng buộc không thể kiểm tra đầy đủ bằng CHECK trong câu lệnh tạo bảng, vì cần đối chiếu dữ liệu giữa nhiều bảng khác nhau. Do đó, nhóm sử dụng trigger để kiểm tra các ràng buộc nghiệp vụ phức tạp và tự động cập nhật các thuộc tính dẫn xuất khi dữ liệu thay đổi.

Trong phần này, nhóm trình bày các trigger chính đại diện cho phân hệ phía Khách hàng gồm:

- `trg_Check_ProductReview_Condition`: kiểm tra điều kiện khách hàng chỉ được đánh giá sản phẩm đã mua trong đơn hàng thanh toán thành công.
- `trg_UpdateOrderTotal_FromOrderDetail`: cập nhật thuộc tính dẫn xuất `TotalAmount` của bảng `ORDER` khi chi tiết đơn hàng thay đổi.
- `trg_UpdateOrderTotal_FromApply`: trigger hỗ trợ cập nhật lại `TotalAmount` khi voucher được áp dụng, cập nhật hoặc xóa khỏi đơn hàng.

Ngoài ra, nhóm còn xây dựng một số trigger hỗ trợ khác như `trg_Check_OrderReview_Condition`, `trg_Review_PreventUpdatePersonID`, `trg_Check_Apply_Rules`, `trg_Check_GuestCustomer_NoPoints`, `trg_Check_ETicket_OneCheckinOnly`, `trg_Check_RefundExchange_NoCheckedInTicket` và `trg_PointHistory_UpdateCurrentPoints`. Các trigger này giúp hệ thống kiểm soát tốt hơn các nghiệp vụ liên quan đến đánh giá, voucher, điểm

tích lũy, vé điện tử và yêu cầu hoàn/đổi vé.

#### 4.2.1 Trigger kiểm tra điều kiện đánh giá sản phẩm

Trigger `trg_Check_ProductReview_Condition` được dùng để kiểm tra ràng buộc nghiệp vụ trên bảng `PRODUCT_REVIEW`. Ràng buộc được chọn là: khách hàng chỉ được đánh giá sản phẩm nếu khách hàng đó đã từng mua sản phẩm trong một đơn hàng của chính mình và đơn hàng đó đã được thanh toán thành công.

Ràng buộc này không thể kiểm tra trực tiếp bằng `CHECK` vì cần truy vấn và đối chiếu dữ liệu từ nhiều bảng khác nhau, bao gồm `PRODUCT_REVIEW`, `REVIEW`, `ORDER`, `PAYMENT_TRANSACTION`, `ORDER_DETAIL` và `PRODUCT_DETAIL`. Do đó, nhóm sử dụng trigger để kiểm tra sau thao tác thêm hoặc cập nhật dữ liệu trong bảng `PRODUCT_REVIEW`.

Các thao tác có thể làm vi phạm ràng buộc gồm:

- **INSERT**: thêm một đánh giá sản phẩm cho sản phẩm mà khách hàng chưa từng mua.
- **UPDATE**: cập nhật mã sản phẩm hoặc mã đánh giá làm cho đánh giá không còn thỏa điều kiện khách hàng đã mua sản phẩm.

```
1 CREATE OR ALTER TRIGGER trg_Check_ProductReview_Condition
2 ON PRODUCT_REVIEW
3 AFTER INSERT, UPDATE
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7
8     IF EXISTS (
9         SELECT 1
10        FROM inserted i
11        JOIN REVIEW r ON i.ReviewID = r.ReviewID
12        WHERE NOT EXISTS (
13            SELECT 1
14            FROM [ORDER] o
15            JOIN PAYMENT_TRANSACTION p
16              ON p.OrderID = o.OrderID
17              AND p.PaymentTransactionPaymentStatus = N'Success'
18            JOIN ORDER_DETAIL od
19              ON od.OrderID = o.OrderID
20            JOIN PRODUCT_DETAIL pd
21              ON pd.OrderDetailID = od.OrderDetailID
22            WHERE o.PersonID = r.PersonID
23              AND pd.ProductID = i.ProductID
24        )
25    )
26 BEGIN
27     RAISERROR (N'Khách hàng chỉ được gửi PRODUCT_REVIEW cho sản phẩm đã mua trong đơn
28     thanh toán thành công của chính mình.', 16, 1);
29     ROLLBACK TRANSACTION;
30     RETURN;
31 END;
32 GO
```

Listing 4.3: Trigger kiểm tra khách hàng chỉ được đánh giá sản phẩm đã mua

Trong trigger này, bảng tạm `inserted` được sử dụng để lấy các dòng đánh giá sản phẩm vừa được thêm hoặc cập nhật. Từ mã đánh giá `ReviewID`, trigger xác định người viết đánh giá thông qua bảng `REVIEW`. Sau đó, trigger kiểm tra xem người này có đơn hàng nào đã thanh toán thành công và có chứa sản phẩm đang được đánh giá hay không.

Nếu không tồn tại đơn hàng thỏa điều kiện, trigger sẽ phát sinh lỗi và rollback giao dịch hiện tại. Nhờ đó, hệ thống ngăn chặn trường hợp khách hàng đánh giá sản phẩm mà mình chưa từng mua hoặc đơn hàng chưa thanh toán thành công.

#### 4.2.2 Các trigger hỗ trợ cho nghiệp vụ đánh giá

Để đảm bảo nghiệp vụ đánh giá được kiểm soát đầy đủ hơn, nhóm xây dựng thêm hai trigger hỗ trợ là `trg_Check_OrderReview_Condition` và `trg_Review_PreventUpdatePersonID`.

Trigger `trg_Check_OrderReview_Condition` được đặt trên bảng `ORDER_REVIEW`. Trigger này kiểm tra rằng chỉ chủ đơn hàng đã thanh toán thành công mới được gửi đánh giá cho đơn hàng đó. Các thao tác có thể làm vi phạm ràng buộc là `INSERT` hoặc `UPDATE` dữ liệu trong bảng `ORDER_REVIEW`.

```
1 CREATE OR ALTER TRIGGER trg_Check_OrderReview_Condition
2 ON ORDER_REVIEW
3 AFTER INSERT, UPDATE
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7
8     IF EXISTS (
9         SELECT 1
10        FROM inserted i
11        JOIN REVIEW r ON i.ReviewID = r.ReviewID
12        JOIN [ORDER] o ON i.OrderID = o.OrderID
13        WHERE r.PersonID <> o.PersonID
14        OR NOT EXISTS (
15            SELECT 1
16            FROM PAYMENT_TRANSACTION p
17            WHERE p.OrderID = o.OrderID
18            AND p.PaymentTransactionPaymentStatus = N'Success'
19        )
20    )
21    BEGIN
22        THROW 50004,
23            N'Chỉ chủ đơn hàng đã thanh toán thành công mới được gửi ORDER_REVIEW.',
24            1;
25    END;
26 END;
27 GO
```

Listing 4.4: Trigger kiểm tra điều kiện đánh giá đơn hàng

Trigger `trg_Review_PreventUpdatePersonID` được đặt trên bảng `REVIEW`. Trigger này không cho phép thay đổi người viết đánh giá sau khi đánh giá đã được tạo. Đây là trigger hỗ trợ quan trọng vì nếu cho phép sửa `PersonID`, người dùng có thể tạo đánh giá hợp lệ ban đầu rồi đổi sang khách hàng khác, làm sai lệch điều kiện kiểm tra của các trigger đánh giá đơn hàng và đánh giá sản phẩm.

```
1 CREATE OR ALTER TRIGGER trg_Review_PreventUpdatePersonID
2 ON REVIEW
```

```

3 AFTER UPDATE
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7
8     IF EXISTS (
9         SELECT 1
10        FROM inserted i
11        INNER JOIN deleted d
12            ON i.ReviewID = d.ReviewID
13        WHERE i.PersonID <> d.PersonID
14    )
15    BEGIN
16        THROW 50003,
17            N'Không được phép thay đổi người viết review sau khi review đã được tạo.',
18            1;
19    END
20 END;
21 GO

```

Listing 4.5: Trigger không cho thay đổi người viết review

Như vậy, bộ trigger liên quan đến đánh giá giúp hệ thống đảm bảo rằng đánh giá chỉ được tạo bởi đúng khách hàng đã thực hiện giao dịch hợp lệ. Điều này giúp dữ liệu đánh giá phản ánh đúng trải nghiệm thực tế của khách hàng và tránh tình trạng đánh giá sai hoặc đánh giá không có căn cứ.

### 4.2.3 Trigger cập nhật tổng tiền đơn hàng

Thuộc tính dẫn xuất được chọn là `TotalAmount` trong bảng `ORDER`. Thuộc tính này lưu tổng số tiền cuối cùng của một đơn hàng sau khi tính tổng các chi tiết đơn hàng và trừ đi tổng số tiền giảm giá từ voucher.

Giá trị `TotalAmount` được tính theo công thức:

$$\text{TotalAmount} = \sum \text{OrderDetailSubtotal} - \sum \text{DiscountAmount}$$

Trong đó, `OrderDetailSubtotal` lấy từ bảng `ORDER_DETAIL`, còn `DiscountAmount` lấy từ bảng `APPLY`. Vì `TotalAmount` phụ thuộc vào dữ liệu của nhiều bảng nên không nên nhập thủ công. Thay vào đó, nhóm sử dụng trigger để tự động tính lại giá trị này khi dữ liệu liên quan thay đổi.

Trigger `trg_UpdateOrderTotal_FromOrderDetail` được đặt trên bảng `ORDER_DETAIL`. Các thao tác có thể làm thay đổi `TotalAmount` gồm:

- INSERT: thêm một dòng chi tiết đơn hàng mới.
- UPDATE: cập nhật số lượng, đơn giá hoặc thành tiền của chi tiết đơn hàng.
- DELETE: xóa một dòng chi tiết đơn hàng khỏi đơn hàng.

```

1 CREATE OR ALTER TRIGGER trg_UpdateOrderTotal_FromOrderDetail
2 ON ORDER_DETAIL
3 AFTER INSERT, UPDATE, DELETE
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7

```

```
8      ;WITH AffectedOrders AS (  
9          SELECT OrderID FROM inserted  
10         UNION  
11         SELECT OrderID FROM deleted  
12     ),  
13     DetailTotal AS (  
14         SELECT  
15             od.OrderID,  
16             SUM(od.OrderDetailSubtotal) AS DetailSum  
17         FROM ORDER_DETAIL od  
18         INNER JOIN AffectedOrders ao ON od.OrderID = ao.OrderID  
19         GROUP BY od.OrderID  
20     ),  
21     DiscountTotal AS (  
22         SELECT  
23             a.OrderID,  
24             SUM(a.DiscountAmount) AS DiscountSum  
25         FROM APPLY a  
26         INNER JOIN AffectedOrders ao ON a.OrderID = ao.OrderID  
27         GROUP BY a.OrderID  
28     )  
29     UPDATE o  
30     SET o.TotalAmount = CAST(  
31         COALESCE(dt.DetailSum, 0) - COALESCE(ap.DiscountSum, 0)  
32         AS INT  
33     )  
34     FROM [ORDER] o  
35     INNER JOIN AffectedOrders ao ON o.OrderID = ao.OrderID  
36     LEFT JOIN DetailTotal dt ON o.OrderID = dt.OrderID  
37     LEFT JOIN DiscountTotal ap ON o.OrderID = ap.OrderID;  
38 END;  
39 GO
```

Listing 4.6: Trigger cập nhật tổng tiền đơn hàng khi ORDER\_DETAIL thay đổi  
Trong trigger này, nhóm sử dụng hai bảng tạm `inserted` và `deleted` để xác định các đơn hàng bị ảnh hưởng. Việc sử dụng cả hai bảng tạm là cần thiết vì thao tác UPDATE hoặc DELETE có thể làm thay đổi dữ liệu cũ, còn thao tác INSERT tạo ra dữ liệu mới.

Sau khi xác định được các đơn hàng bị ảnh hưởng, trigger tính tổng giá trị các dòng chi tiết đơn hàng trong bảng ORDER\_DETAIL, đồng thời tính tổng số tiền giảm giá trong bảng APPLY. Cuối cùng, trigger cập nhật lại TotalAmount của từng đơn hàng theo công thức đã nêu.

#### 4.2.4 Trigger hỗ trợ cập nhật tổng tiền khi áp dụng voucher

Bên cạnh việc thay đổi chi tiết đơn hàng, tổng tiền đơn hàng cũng có thể thay đổi khi khách hàng áp dụng, cập nhật hoặc xóa voucher. Vì vậy, nhóm xây dựng thêm trigger `trg_UpdateOrderTotal_FromApply` trên bảng APPLY để hỗ trợ cập nhật lại TotalAmount khi dữ liệu voucher của đơn hàng thay đổi.

Các thao tác trên bảng APPLY có thể làm thay đổi tổng tiền đơn hàng gồm:

- INSERT: áp dụng thêm voucher cho đơn hàng.
- UPDATE: thay đổi số tiền giảm giá của voucher đã áp dụng.
- DELETE: xóa voucher khỏi đơn hàng.

```
1 CREATE OR ALTER TRIGGER trg_UpdateOrderTotal_FromApply
2 ON APPLY
3 AFTER INSERT, UPDATE, DELETE
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7
8     ;WITH AffectedOrders AS (
9         SELECT OrderID FROM inserted
10        UNION
11        SELECT OrderID FROM deleted
12    ),
13    DetailTotal AS (
14        SELECT
15            od.OrderID,
16            SUM(od.OrderDetailSubtotal) AS DetailSum
17        FROM ORDER_DETAIL od
18        INNER JOIN AffectedOrders ao ON od.OrderID = ao.OrderID
19        GROUP BY od.OrderID
20    ),
21    DiscountTotal AS (
22        SELECT
23            a.OrderID,
24            SUM(a.DiscountAmount) AS DiscountSum
25        FROM APPLY a
26        INNER JOIN AffectedOrders ao ON a.OrderID = ao.OrderID
27        GROUP BY a.OrderID
28    )
29    UPDATE o
30    SET o.TotalAmount = CAST(
31        COALESCE(dt.DetailSum, 0) - COALESCE(ap.DiscountSum, 0)
32        AS INT
33    )
34    FROM [ORDER] o
35    INNER JOIN AffectedOrders ao ON o.OrderID = ao.OrderID
36    LEFT JOIN DetailTotal dt ON o.OrderID = dt.OrderID
37    LEFT JOIN DiscountTotal ap ON o.OrderID = ap.OrderID;
38 END;
39 GO
```

Listing 4.7: Trigger cập nhật tổng tiền đơn hàng khi APPLY thay đổi  
Trigger này có vai trò hỗ trợ cho trigger `trg_UpdateOrderTotal_FromOrderDetail`. Nếu chỉ có trigger trên bảng `ORDER_DETAIL`, hệ thống chỉ cập nhật tổng tiền khi chi tiết đơn hàng thay đổi, nhưng không tự động cập nhật khi voucher thay đổi. Do đó, trigger trên bảng `APPLY` là cần thiết để đảm bảo `TotalAmount` luôn phản ánh đúng số tiền cuối cùng của đơn hàng.

#### 4.2.5 Trigger kiểm tra điều kiện áp dụng voucher

Trigger `trg_Check_Apply_Rules` được sử dụng để kiểm tra các ràng buộc nghiệp vụ khi áp dụng voucher cho đơn hàng. Trigger này đóng vai trò hỗ trợ cho hai trigger cập nhật tổng tiền đơn hàng, vì trước khi cập nhật `TotalAmount`, hệ thống cần đảm bảo dữ liệu giảm giá là hợp lệ.

Trigger này kiểm tra hai điều kiện chính:

- Voucher phải đang ở trạng thái **Active** và chưa hết hạn tại thời điểm áp dụng.
- Tổng số tiền giảm giá của đơn hàng không được vượt quá tổng giá trị gốc của các chi tiết đơn hàng.

Các thao tác có thể làm vi phạm ràng buộc gồm:

- INSERT: áp dụng voucher đã hết hạn hoặc tạo mức giảm giá vượt quá giá trị đơn hàng.
- UPDATE: cập nhật voucher hoặc số tiền giảm giá làm cho đơn hàng không còn hợp lệ.

```
1 CREATE OR ALTER TRIGGER trg_Check_Apply_Rules
2 ON APPLY
3 AFTER INSERT, UPDATE
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7
8     IF EXISTS (
9         SELECT 1
10        FROM inserted i
11        JOIN VOUCHER v ON i.VoucherID = v.VoucherID
12        JOIN [ORDER] o ON i.OrderID = o.OrderID
13        WHERE v.VoucherStatus <> N'Active'
14              OR CAST(o.OrderDate AS DATETIME) > v.VoucherExpiresAt
15    )
16    BEGIN
17        RAISERROR (N'Voucher không còn hiệu lực hoặc đã hết hạn tại thời điểm áp dụng.',
18        16, 1);
19        ROLLBACK TRANSACTION;
20        RETURN;
21    END;
22
23    IF EXISTS (
24        SELECT 1
25        FROM (
26            SELECT
27                a.OrderID,
28                SUM(a.DiscountAmount) AS TotalDiscount
29            FROM APPLY a
30            JOIN (
31                SELECT OrderID FROM inserted
32            ) ao ON a.OrderID = ao.OrderID
33            GROUP BY a.OrderID
34        ) d
35        JOIN (
36            SELECT
37                od.OrderID,
38                SUM(od.OrderDetailSubtotal) AS GrossAmount
39            FROM ORDER_DETAIL od
40            JOIN (
41                SELECT OrderID FROM inserted
42            ) ao ON od.OrderID = ao.OrderID
43            GROUP BY od.OrderID
44        ) g
45        ON d.OrderID = g.OrderID
46        WHERE d.TotalDiscount > g.GrossAmount
47    )
48    BEGIN
49        RAISERROR (N'Tổng DiscountAmount của đơn hàng không được vượt quá giá trị gốc của
50        đơn.', 16, 1);
51        ROLLBACK TRANSACTION;
52        RETURN;
53    END;
```

```
52 END ;  
53 GO
```

Listing 4.8: Trigger kiểm tra điều kiện áp dụng voucher

Nhờ trigger này, dữ liệu trong bảng **APPLY** được kiểm soát trước khi ảnh hưởng đến tổng tiền đơn hàng. Nếu voucher không hợp lệ hoặc tổng giảm giá vượt quá giá trị đơn hàng, giao dịch sẽ bị rollback và **TotalAmount** của đơn hàng sẽ không bị cập nhật sai.

#### 4.2.6 Trigger cập nhật điểm hiện tại của khách hàng thành viên

Ngoài tổng tiền đơn hàng, hệ thống còn có thuộc tính dẫn xuất **CurrentPoints** trong bảng **REGISTERED\_CUSTOMER**. Thuộc tính này lưu tổng số điểm hiện tại của khách hàng thành viên và được tính dựa trên lịch sử thay đổi điểm trong bảng **POINT\_HISTORY**.

Giá trị **CurrentPoints** được tính theo công thức:

$$\text{CurrentPoints} = \sum \text{PointChange}$$

Các thao tác trên bảng **POINT\_HISTORY** có thể làm thay đổi **CurrentPoints** gồm:

- INSERT: thêm một dòng lịch sử cộng hoặc trừ điểm.
- UPDATE: thay đổi số điểm, khách hàng hoặc lý do phát sinh điểm.
- DELETE: xóa một dòng lịch sử điểm.

```
1 CREATE OR ALTER TRIGGER trg_PointHistory_UpdateCurrentPoints  
2 ON POINT_HISTORY  
3 AFTER INSERT, UPDATE, DELETE  
4 AS  
5 BEGIN  
6     SET NOCOUNT ON;  
7  
8     ;WITH AffectedPerson AS (  
9         SELECT PersonID  
10        FROM inserted  
11        WHERE PersonID IS NOT NULL  
12  
13        UNION  
14  
15        SELECT PersonID  
16        FROM deleted  
17        WHERE PersonID IS NOT NULL  
18    ),  
19    PointSummary AS (  
20        SELECT  
21            ap.PersonID,  
22            ISNULL(SUM(ph.PointChange), 0) AS TotalPoints  
23        FROM AffectedPerson ap  
24        LEFT JOIN POINT_HISTORY ph  
25            ON ph.PersonID = ap.PersonID  
26        GROUP BY ap.PersonID  
27    )  
28    UPDATE rc  
29    SET rc.CurrentPoints = ps.TotalPoints
```

```

30 FROM REGISTERED_CUSTOMER rc
31 INNER JOIN PointSummary ps
32     ON ps.PersonID = rc.PersonID;
33 END;
34 GO

```

Listing 4.9: Trigger cập nhật điểm hiện tại của khách hàng thành viên

Trigger này sử dụng `inserted` và `deleted` để xác định các khách hàng bị ảnh hưởng khi lịch sử điểm thay đổi. Sau đó, trigger tính lại tổng điểm từ bảng `POINT_HISTORY` và cập nhật vào `REGISTERED_CUSTOMER.CurrentPoints`. Cách tính lại toàn bộ tổng điểm giúp tránh sai lệch dữ liệu trong trường hợp có thao tác sửa hoặc xóa lịch sử điểm.

Để hỗ trợ trigger này, nhóm xây dựng thêm trigger `trg_Check_GuestCustomer_NoPoints`. Trigger hỗ trợ này đảm bảo chỉ khách hàng thành viên mới được có lịch sử điểm, còn khách vắng lai không được tích điểm.

```

1 CREATE OR ALTER TRIGGER trg_Check_GuestCustomer_NoPoints
2 ON POINT_HISTORY
3 AFTER INSERT, UPDATE
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7
8     IF EXISTS (
9         SELECT 1
10        FROM inserted i
11        JOIN GUEST_CUSTOMER g ON i.PersonID = g.PersonID
12    )
13    BEGIN
14        RAISERROR (N'Khách vắng lai không được tích điểm hoặc lưu lịch sử điểm.', 16, 1);
15        ROLLBACK TRANSACTION;
16        RETURN;
17    END;
18
19    IF EXISTS (
20        SELECT 1
21        FROM inserted i
22        LEFT JOIN REGISTERED_CUSTOMER r ON i.PersonID = r.PersonID
23        WHERE r.PersonID IS NULL
24    )
25    BEGIN
26        RAISERROR (N'Chỉ khách hàng thành viên mới được phép có lịch sử điểm.', 16, 1);
27        ROLLBACK TRANSACTION;
28        RETURN;
29    END;
30 END;
31 GO

```

Listing 4.10: Trigger kiểm tra khách vắng lai không được tích điểm

Như vậy, `trg_Check_GuestCustomer_NoPoints` có vai trò kiểm tra dữ liệu đầu vào của bảng `POINT_HISTORY`, còn `trg_PointHistory_UpdateCurrentPoints` có vai trò tính lại điểm hiện tại sau khi dữ liệu lịch sử điểm thay đổi. Hai trigger này bổ trợ cho nhau để đảm bảo dữ liệu điểm tích lũy của khách hàng luôn chính xác và hợp lệ.

#### 4.2.7 Các trigger hỗ trợ khác trong phân hệ Khách hàng

Ngoài các trigger chính đã trình bày ở trên, nhóm còn xây dựng một số trigger hỗ trợ khác nhằm kiểm soát các nghiệp vụ đặc thù trong hệ thống rạp chiếu phim.

Trigger `trg_Check_ETicket_OneCheckinOnly` được đặt trên bảng `E_TICKET`. Trigger này đảm bảo rằng nếu vé điện tử có trạng thái `Checked-in` thì bắt buộc phải có thời gian check-in. Đồng thời, nếu vé đã check-in thì không được check-in lại hoặc đổi ngược trạng thái. Trigger này giúp ngăn chặn việc sử dụng một vé nhiều lần.

Trigger `trg_Check_RefundExchange_NoCheckedInTicket` được đặt trên bảng `REFUND_EXCHANGE_REQUEST`. Trigger này kiểm tra rằng vé đã check-in thì không được tạo hoặc cập nhật yêu cầu hoàn/đổi. Điều này phù hợp với nghiệp vụ thực tế, vì vé đã được sử dụng để vào rạp thì không còn đủ điều kiện hoàn tiền hoặc đổi vé.

Các trigger hỗ trợ này không phải là trigger chính được chọn để trình bày cho yêu cầu 2.2, nhưng chúng giúp hệ thống đầy đủ hơn về mặt nghiệp vụ và tăng tính nhất quán của dữ liệu trong quá trình vận hành.

#### 4.2.8 Nhận xét

Các trigger trong phân hệ phía Khách hàng được xây dựng nhằm kiểm soát những nghiệp vụ quan trọng như đánh giá sản phẩm, đánh giá đơn hàng, áp dụng voucher, tính tổng tiền đơn hàng, cập nhật điểm tích lũy, kiểm soát vé check-in và xử lý yêu cầu hoàn/đổi vé.

Đối với yêu cầu kiểm tra ràng buộc nghiệp vụ, trigger `trg_Check_ProductReview_Condition` thể hiện rõ việc sử dụng trigger để kiểm tra dữ liệu liên quan đến nhiều bảng, đảm bảo khách hàng chỉ được đánh giá sản phẩm đã mua trong đơn hàng thanh toán thành công.

Đối với yêu cầu cập nhật thuộc tính dẫn xuất, hai trigger `trg_UpdateOrderTotal_FromOrderDetail` và `trg_UpdateOrderTotal_FromApply` được sử dụng để tự động tính lại `ORDER.TotalAmount`. Ngoài ra, trigger `trg_PointHistory_UpdateCurrentPoints` cũng được xây dựng để cập nhật thuộc tính dẫn xuất `REGISTERED_CUSTOMER.CurrentPoints`.

Các trigger đều được viết theo hướng xử lý nhiều dòng dữ liệu thông qua bảng tạm `inserted` và `deleted`. Điều này phù hợp với cách hoạt động của SQL Server, vì một câu lệnh `INSERT`, `UPDATE` hoặc `DELETE` có thể tác động đến nhiều dòng cùng lúc. Nhờ đó, hệ thống có thể hạn chế dữ liệu không hợp lệ, tự động cập nhật dữ liệu dẫn xuất và đảm bảo tính nhất quán giữa các bảng liên quan.

# CHƯƠNG 5. THỦ TỤC TRUY VẤN DỮ LIỆU

## 5.1 Các thủ tục truy vấn của phân hệ sự kiện

Sau khi hiện thực các thủ tục thêm, sửa và xóa dữ liệu trên bảng **EVENT**, nhóm tiếp tục xây dựng hai thủ tục dùng để truy vấn và hiển thị dữ liệu. Các thủ tục này không làm thay đổi dữ liệu trong cơ sở dữ liệu, mà chỉ phục vụ việc tra cứu, lọc và thống kê thông tin sự kiện.

Hai thủ tục được hiện thực gồm:

- **usp\_Event\_List**: hiển thị danh sách sự kiện theo các điều kiện lọc.
- **usp\_Organizer\_Event\_Summary**: thống kê tình hình tổ chức sự kiện theo từng nhà tổ chức.

Cả hai thủ tục đều có tham số đầu vào để người dùng có thể lọc dữ liệu theo nhu cầu. Trong đó, **usp\_Event\_List** liên quan trực tiếp đến bảng **EVENT** đã chọn ở phần thêm, sửa và xóa dữ liệu.

### 5.1.1 Thủ tục hiển thị danh sách sự kiện

Thủ tục **usp\_Event\_List** được dùng để hiển thị danh sách các sự kiện trong hệ thống. Thủ tục này liên kết bảng **EVENT** với bảng **ORGANIZER** để hiển thị thêm thông tin nhà tổ chức của từng sự kiện.

Các tham số của thủ tục gồm:

- **@Keyword**: từ khóa tìm kiếm theo mã sự kiện, tên sự kiện hoặc tên nhà tổ chức.
- **@EventStatus**: trạng thái sự kiện cần lọc.
- **@FromDate**: ngày bắt đầu nhỏ nhất của sự kiện.
- **@ToDate**: ngày bắt đầu lớn nhất của sự kiện.

Nếu một tham số không được truyền vào hoặc có giá trị rỗng, thủ tục sẽ bỏ qua điều kiện lọc tương ứng. Cách làm này giúp thủ tục có thể dùng linh hoạt cho nhiều trường hợp, từ hiển thị toàn bộ danh sách đến tìm kiếm theo một điều kiện cụ thể.

```
1  SELECT
2      EVENT.EventID ,
3      EVENT.EventName ,
4      EVENT.EventStartDate ,
5      EVENT.EventEndDate ,
6      EVENT.EventStatus ,
7      EVENT.ExpectedScale ,
8      EVENT.TotalBudget ,
9      EVENT.ConfirmedSessionCount ,
10     ORGANIZER.OrgName ,
11     ORGANIZER.OrganizerType
12  FROM  EVENT
13  JOIN  ORGANIZER ON EVENT.OrganizerID = ORGANIZER.OrganizerID
14  WHERE (@Keyword IS NULL
```

```

15 OR EVENT.EventID LIKE '%' + @Keyword + '%'
16 OR EVENT.EventName LIKE N'%' + @Keyword + N'%'
17 OR ORGANIZER.OrgName LIKE N'%' + @Keyword + N'%'
18 AND (@EventStatus IS NULL OR EVENT.EventStatus = @EventStatus)
19 AND (@FromDate IS NULL OR EVENT.EventStartDate >= @FromDate)
20 AND (@ToDate IS NULL OR EVENT.EventStartDate <= @ToDate)
21 ORDER BY EVENT.EventStartDate ASC, EVENT.EventName ASC;

```

Listing 5.1: Phần truy vấn chính trong thủ tục usp\_Event\_List

Thủ tục này đáp ứng yêu cầu truy vấn từ hai bảng trở lên, có mệnh đề WHERE và có ORDER BY. Kết quả trả về có thể dùng cho màn hình danh sách sự kiện trong ứng dụng, trong đó người dùng có thể tìm kiếm theo từ khóa, lọc theo trạng thái hoặc lọc theo khoảng ngày.

### 5.1.2 Thủ tục thống kê sự kiện theo nhà tổ chức

Thủ tục usp\_Organizer\_Event\_Summary được dùng để thống kê số lượng sự kiện và chi phí thuê phòng đã xác nhận theo từng nhà tổ chức. Thủ tục này liên kết ba bảng ORGANIZER, EVENT và SESSION.

Các tham số của thủ tục gồm:

- @EventStatus: lọc sự kiện theo trạng thái.
- @BudgetFloor: chỉ xét các sự kiện có ngân sách từ mức này trở lên.
- @MinEventCount: chỉ lấy các nhà tổ chức có số lượng sự kiện đạt mức tối thiểu.
- @MinConfirmedRentalFee: chỉ lấy các nhà tổ chức có tổng phí thuê đã xác nhận đạt mức tối thiểu.

Khác với thủ tục danh sách sự kiện, thủ tục này có sử dụng các hàm tổng hợp như COUNT và SUM. Dữ liệu được nhóm theo từng nhà tổ chức bằng GROUP BY. Sau khi nhóm dữ liệu, thủ tục tiếp tục dùng HAVING để lọc các nhóm thỏa điều kiện thống kê.

```

1 SELECT
2     ORGANIZER.OrganizerID,
3     ORGANIZER.OrgName,
4     ORGANIZER.OrganizerType,
5     COUNT(DISTINCT EVENT.EventID) AS EventCount,
6     SUM(ISNULL(EVENT.TotalBudget, 0)) AS TotalPlannedBudget,
7     SUM(CASE
8         WHEN SESSION.SessionStatus = N'Da xac nhan'
9         THEN ISNULL(SESSION.RentalFee, 0)
10        ELSE 0
11    END) AS TotalConfirmedRentalFee,
12    SUM(CASE
13        WHEN SESSION.SessionStatus = N'Da xac nhan'
14        THEN 1
15        ELSE 0
16    END) AS TotalConfirmedSessionCount
17 FROM ORGANIZER
18 JOIN EVENT ON ORGANIZER.OrganizerID = EVENT.OrganizerID
19 LEFT JOIN SESSION ON EVENT.EventID = SESSION.EventID
20 WHERE (@EventStatus IS NULL OR EVENT.EventStatus = @EventStatus)
21 AND ISNULL(EVENT.TotalBudget, 0) >= @BudgetFloor
22 GROUP BY ORGANIZER.OrganizerID, ORGANIZER.OrgName, ORGANIZER.OrganizerType
23 HAVING COUNT(DISTINCT EVENT.EventID) >= @MinEventCount
24 AND SUM(CASE
25     WHEN SESSION.SessionStatus = N'Da xac nhan'

```

```
26         THEN ISNULL(SESSION.RentalFee, 0)
27         ELSE 0
28     END) >= @MinConfirmedRentalFee
29 ORDER BY TotalConfirmedRentalFee DESC, EventCount DESC, ORGANIZER.OrgName ASC;
```

Listing 5.2: Phần truy vấn chính trong thủ tục usp\_Organizer\_Event\_Summary  
Thủ tục này đáp ứng yêu cầu truy vấn có liên kết từ hai bảng trở lên, có WHERE, có hàm tổng hợp, có GROUP BY, có HAVING và có ORDER BY. Kết quả truy vấn giúp đánh giá nhà tổ chức nào có nhiều sự kiện, tổng ngân sách dự kiến cao và tổng phí thuê đã xác nhận lớn.

## 5.2 Các thủ tục truy vấn của phân hệ Khách hàng

Sau khi hiện thực các thủ tục thêm, sửa và xóa dữ liệu trên bảng PRODUCT, nhóm tiếp tục xây dựng thủ tục dùng để truy vấn và thống kê dữ liệu liên quan đến sản phẩm. Thủ tục này không làm thay đổi dữ liệu trong cơ sở dữ liệu, mà chỉ phục vụ việc hiển thị, lọc và thống kê thông tin bán hàng của sản phẩm trong hệ thống rạp chiếu phim.

Thủ tục được hiện thực là:

- **sp\_GetProductSalesSummary**: thống kê số lượng bán và doanh thu của từng sản phẩm trong một khoảng thời gian.

Thủ tục này có tham số đầu vào để người dùng có thể lọc dữ liệu theo loại sản phẩm, khoảng thời gian đặt hàng, số lượng bán tối thiểu và doanh thu tối thiểu. Đây là thủ tục truy vấn có sử dụng liên kết nhiều bảng, hàm tổng hợp, GROUP BY, HAVING, WHERE và ORDER BY, phù hợp với yêu cầu của bài tập lớn.

### 5.2.1 Thủ tục thống kê doanh thu sản phẩm

Thủ tục **sp\_GetProductSalesSummary** được dùng để thống kê tình hình bán sản phẩm trong hệ thống rạp chiếu phim. Các sản phẩm có thể bao gồm bắp, nước, combo hoặc các sản phẩm khác được bán kèm trong quá trình khách hàng mua vé xem phim.

Thủ tục này lấy dữ liệu từ các bảng:

- **PRODUCT**: lưu thông tin sản phẩm.
- **PRODUCT\_DETAIL**: lưu chi tiết sản phẩm trong đơn hàng.
- **ORDER\_DETAIL**: lưu số lượng, đơn giá và thành tiền của từng dòng đơn hàng.
- **ORDER**: lưu thông tin đơn hàng và ngày đặt hàng.
- **REFUND\_EXCHANGE\_REQUEST**: lưu thông tin yêu cầu hoàn tiền hoặc đổi trả.

Các tham số của thủ tục gồm:

- **@ProductType**: loại sản phẩm cần thống kê. Nếu tham số này bằng NULL thì thủ tục thống kê tất cả loại sản phẩm.
- **@FromDate**: ngày bắt đầu của khoảng thời gian thống kê.
- **@ToDate**: ngày kết thúc của khoảng thời gian thống kê.
- **@MinTotalQuantity**: số lượng bán tối thiểu dùng để lọc kết quả.

- @MinTotalRevenue: doanh thu tối thiểu dùng để lọc kết quả.

Trước khi thực hiện truy vấn, thủ tục chuẩn hóa tham số @ProductType bằng cách loại bỏ khoảng trắng thừa. Nếu người dùng truyền vào chuỗi rỗng, thủ tục sẽ xem như không lọc theo loại sản phẩm. Ngoài ra, thủ tục cũng kiểm tra dữ liệu đầu vào để đảm bảo ngày bắt đầu không lớn hơn ngày kết thúc, số lượng tối thiểu không được âm và doanh thu tối thiểu không được âm.

```
1 SET @ProductType = NULLIF(LTRIM(RTRIM(@ProductType)), N'');
2
3 IF @FromDate IS NOT NULL
4     AND @ToDate IS NOT NULL
5     AND @FromDate > @ToDate
6     THROW 50301, N'Ngày bắt đầu không được lớn hơn ngày kết thúc.', 1;
7
8 IF @MinTotalQuantity < 0
9     THROW 50302, N'Số lượng tối thiểu không được âm.', 1;
10
11 IF @MinTotalRevenue < 0
12     THROW 50303, N'Doanh thu tối thiểu không được âm.', 1;
```

Listing 5.3: Kiểm tra tham số đầu vào trong thủ tục sp\_GetProductSalesSummary  
Phần truy vấn chính của thủ tục sử dụng biểu thức bảng chung SalesBase để lấy dữ liệu bán hàng ban đầu. Trong phần này, thủ tục liên kết các bảng PRODUCT, PRODUCT\_DETAIL, ORDER\_DETAIL, ORDER và REFUND\_EXCHANGE\_REQUEST. Mệnh đề WHERE được sử dụng để lọc dữ liệu theo loại sản phẩm và khoảng thời gian đặt hàng.

Đối với tiền hoàn, thủ tục chỉ tính hoàn tiền khi yêu cầu có RequestType = N'Refund' và RequestStatus = N'Approved'. Nếu số tiền hoàn lớn hơn giá trị dòng chi tiết đơn hàng, thủ tục chỉ trừ tối đa bằng giá trị của dòng chi tiết đó để tránh trường hợp doanh thu sản phẩm bị âm.

```
1 ;WITH SalesBase AS
2 (
3     SELECT
4         p.ProductID,
5         p.ProductName,
6         p.ProductType,
7         p.ProductStatus,
8         o.OrderID,
9         od.OrderDetailQuantity,
10        od.OrderDetailSubtotal,
11        CASE
12            WHEN rer.RequestID IS NOT NULL THEN
13                CASE
14                    WHEN ISNULL(rer.RequestActualRefundAmount, 0) > od.
15                        OrderDetailSubtotal
16                        THEN od.OrderDetailSubtotal
17                    ELSE ISNULL(rer.RequestActualRefundAmount, 0)
18                END
19            ELSE 0
20        END AS RefundAmount
21    FROM PRODUCT p
22    JOIN PRODUCT_DETAIL pd
23        ON pd.ProductID = p.ProductID
24    JOIN ORDER_DETAIL od
25        ON od.OrderDetailID = pd.OrderDetailID
26    JOIN [ORDER] o
```

```

26         ON o.OrderID = od.OrderID
27     LEFT JOIN REFUND_EXCHANGE_REQUEST rer
28         ON rer.RequestID = od.RequestID
29         AND rer.RequestType = N'Refund'
30         AND rer.RequestStatus = N'Approved'
31     WHERE (@ProductType IS NULL OR p.ProductType = @ProductType)
32         AND (@FromDate IS NULL OR o.OrderDate >= @FromDate)
33         AND (@ToDate IS NULL OR o.OrderDate <= @ToDate)
34 )

```

Listing 5.4: Phần lấy dữ liệu bán hàng trong thủ tục `sp_GetProductSalesSummary`. Sau khi lấy dữ liệu cơ sở, thủ tục thực hiện thống kê theo từng sản phẩm. Mệnh đề `GROUP BY` được dùng để gom các dòng bán hàng của cùng một sản phẩm thành một nhóm. Các hàm tổng hợp như `COUNT` và `SUM` được sử dụng để tính tổng số đơn hàng, tổng số lượng bán, doanh thu gốc, tổng tiền hoàn và doanh thu sau hoàn tiền.

```

1  SELECT
2      ProductID,
3      ProductName,
4      ProductType,
5      ProductStatus,
6      COUNT(DISTINCT OrderID)           AS TotalOrders,
7      SUM(OrderDetailQuantity)          AS TotalQuantitySold,
8      SUM(OrderDetailSubtotal)          AS GrossRevenue,
9      SUM(RefundAmount)                 AS TotalRefundAmount,
10     SUM(OrderDetailSubtotal - RefundAmount) AS TotalRevenue
11 FROM SalesBase
12 GROUP BY
13     ProductID,
14     ProductName,
15     ProductType,
16     ProductStatus
17 HAVING
18     SUM(OrderDetailQuantity) >= @MinTotalQuantity
19     AND SUM(OrderDetailSubtotal - RefundAmount) >= @MinTotalRevenue
20 ORDER BY
21     TotalRevenue DESC,
22     TotalQuantitySold DESC,
23     ProductName ASC;

```

Listing 5.5: Phần thống kê doanh thu sản phẩm

Trong đó:

- `TotalOrders`: tổng số đơn hàng có chứa sản phẩm.
- `TotalQuantitySold`: tổng số lượng sản phẩm đã bán.
- `GrossRevenue`: doanh thu gốc trước khi trừ tiền hoàn.
- `TotalRefundAmount`: tổng số tiền hoàn của sản phẩm.
- `TotalRevenue`: doanh thu thực tế sau khi trừ tiền hoàn.

Mệnh đề `HAVING` được sử dụng để lọc các sản phẩm sau khi đã gom nhóm. Cụ thể, thủ tục chỉ hiển thị các sản phẩm có tổng số lượng bán lớn hơn hoặc bằng `@MinTotalQuantity` và doanh thu sau hoàn tiền lớn hơn hoặc bằng `@MinTotalRevenue`. Cuối cùng, kết quả được sắp xếp bằng `ORDER BY`, ưu tiên sản phẩm có doanh thu sau hoàn tiền cao hơn, sau đó đến số lượng bán và tên sản phẩm.

Thủ tục `sp_GetProductSalesSummary` đáp ứng yêu cầu truy vấn dữ liệu của bài tập lớn vì có liên kết từ nhiều bảng, có mệnh đề `WHERE`, có sử dụng hàm tổng hợp, có `GROUP BY`, có `HAVING` và có `ORDER BY`. Kết quả của thủ tục có thể được sử dụng cho màn hình thống kê sản phẩm trong ứng dụng, giúp người quản lý biết được sản phẩm nào bán chạy, sản phẩm nào mang lại doanh thu cao và số tiền hoàn trả phát sinh trong từng khoảng thời gian.

# CHƯƠNG 6. HÀM

## 6.1 Các hàm của phân hệ sự kiện

Trong phân hệ phía Ráp, nhóm hiện thực hai hàm liên quan đến bảng **EVENT** và **SESSION**. Các hàm này được dùng để tính toán dữ liệu phục vụ việc tra cứu và đánh giá tình trạng tổ chức sự kiện.

Hai hàm được hiện thực gồm:

- **fn\_Event\_TotalConfirmedRentalFee**: tính tổng phí thuê của các phiên sự kiện đã được xác nhận.
- **fn\_Event\_CapacityCoverageLabel**: đánh giá mức độ đáp ứng sức chứa của các phiên sự kiện so với quy mô dự kiến.

Cả hai hàm đều có tham số đầu vào là mã sự kiện **@EventID**. Trước khi tính toán, hàm kiểm tra tham số đầu vào và kiểm tra sự kiện có tồn tại trong bảng **EVENT** hay không. Ngoài ra, các hàm đều sử dụng con trỏ để duyệt qua các dòng dữ liệu trong bảng **SESSION**, từ đó thực hiện tính toán bằng vòng lặp.

### 6.1.1 Hàm tính tổng phí thuê đã xác nhận

Hàm **fn\_Event\_TotalConfirmedRentalFee** được dùng để tính tổng phí thuê của một sự kiện. Hàm chỉ tính các phiên có trạng thái **Đã xác nhận**, vì đây là các phiên đã được chấp nhận tổ chức và có giá trị nghiệp vụ rõ ràng.

Tham số đầu vào của hàm là **@EventID**. Nếu mã sự kiện bị rỗng hoặc không tồn tại trong bảng **EVENT**, hàm trả về **NULL**. Nếu sự kiện tồn tại, hàm duyệt qua các dòng trong bảng **SESSION** có cùng mã sự kiện và có trạng thái **Đã xác nhận**, sau đó cộng dồn giá trị **RentalFee**.

```
1 CREATE FUNCTION fn_Event_TotalConfirmedRentalFee
2 (
3     @EventID VARCHAR(20)
4 )
5 RETURNS DECIMAL(15,2)
6 AS
7 BEGIN
8     DECLARE @Tong DECIMAL(15,2);
9     DECLARE @Tien DECIMAL(12,2);
10
11     SET @Tong = 0;
12     SET @EventID = NULLIF(LTRIM(RTRIM(@EventID)), '');
13
14     IF @EventID IS NULL
15         RETURN NULL;
16
17     IF NOT EXISTS (SELECT 1 FROM EVENT WHERE EventID = @EventID)
18         RETURN NULL;
19
20     DECLARE cur_Tien CURSOR LOCAL FAST_FORWARD FOR
21         SELECT ISNULL(RentalFee, 0)
22         FROM SESSION
23         WHERE EventID = @EventID
```

```

24         AND SessionStatus = N'Da xac nhan'
25     ORDER BY SessionStart, SessionID;
26
27     OPEN cur_Tien;
28     FETCH NEXT FROM cur_Tien INTO @Tien;
29
30     WHILE @@FETCH_STATUS = 0
31     BEGIN
32         SET @Tong = @Tong + ISNULL(@Tien, 0);
33         FETCH NEXT FROM cur_Tien INTO @Tien;
34     END;
35
36     CLOSE cur_Tien;
37     DEALLOCATE cur_Tien;
38
39     RETURN @Tong;
40 END;

```

Listing 6.1: Hàm tính tổng phí thuê đã xác nhận của sự kiện

Trong hàm này, con trỏ `cur_Tien` được dùng để duyệt từng phiên sự kiện thỏa điều kiện. Biến `@Tong` lưu tổng phí thuê và được cập nhật sau mỗi vòng lặp. Kết quả trả về là tổng phí thuê của toàn bộ phiên đã xác nhận thuộc sự kiện cần tính.

### 6.1.2 Hàm đánh giá mức độ đáp ứng sức chứa

Hàm `fn_Event_CapacityCoverageLabel` được dùng để đánh giá tổng sức chứa của các phiên sự kiện so với quy mô dự kiến trong bảng `EVENT`. Hàm này giúp xác định các phiên đã tạo có đủ khả năng phục vụ số lượng khách dự kiến hay chưa.

Tham số đầu vào của hàm là `@EventID`. Nếu mã sự kiện không hợp lệ, hàm trả về thông báo Mã sự kiện không hợp lệ. Nếu không tìm thấy sự kiện trong bảng `EVENT`, hàm trả về thông báo Không tìm thấy sự kiện. Nếu sự kiện chưa có phiên tổ chức hợp lệ, hàm trả về thông báo Chưa có phiên sự kiện.

Hàm chỉ xét các phiên không bị hủy. Với mỗi phiên, hàm lấy giá trị `MaxGuests` và cộng dồn vào tổng sức chứa. Sau đó, hàm tính tỷ lệ đáp ứng theo công thức:

$$Coverage = \frac{TotalCapacity \times 100}{ExpectedScale}$$

Trong đó, `TotalCapacity` là tổng sức chứa của các phiên sự kiện và `ExpectedScale` là quy mô dự kiến của sự kiện.

```

1 CREATE FUNCTION fn_Event_CapacityCoverageLabel
2 (
3     @EventID VARCHAR(20)
4 )
5 RETURNS NVARCHAR(100)
6 AS
7 BEGIN
8     DECLARE @ExpectedScale INT;
9     DECLARE @TotalCapacity INT;
10    DECLARE @OneCapacity INT;
11    DECLARE @Coverage DECIMAL(10,2);
12

```

```
13 SET @TotalCapacity = 0;
14 SET @EventID = NULLIF(LTRIM(RTRIM(@EventID)), '');
15
16 IF @EventID IS NULL
17     RETURN N'Ma su kien khong hop le';
18
19 SELECT @ExpectedScale = ExpectedScale
20 FROM EVENT
21 WHERE EventID = @EventID;
22
23 IF @ExpectedScale IS NULL
24     RETURN N'Khong tim thay su kien';
25
26 DECLARE cur_Capacity CURSOR LOCAL FAST_FORWARD FOR
27     SELECT ISNULL(MaxGuests, 0)
28     FROM SESSION
29     WHERE EventID = @EventID
30           AND SessionStatus <> N'Huy'
31     ORDER BY SessionStart, SessionID;
32
33 OPEN cur_Capacity;
34 FETCH NEXT FROM cur_Capacity INTO @OneCapacity;
35
36 WHILE @@FETCH_STATUS = 0
37 BEGIN
38     SET @TotalCapacity = @TotalCapacity + ISNULL(@OneCapacity, 0);
39     FETCH NEXT FROM cur_Capacity INTO @OneCapacity;
40 END;
41
42 CLOSE cur_Capacity;
43 DEALLOCATE cur_Capacity;
44
45 IF @TotalCapacity = 0
46     RETURN N'Chua co phien su kien';
47
48 SET @Coverage = CAST(@TotalCapacity * 100.0 / NULLIF(@ExpectedScale, 0) AS DECIMAL
49 (10,2));
50
51 IF @Coverage < 70
52     RETURN N'Thap';
53
54 IF @Coverage < 100
55     RETURN N'Gan dat';
56
57 RETURN N'Dat hoac vuot';
58 END;
```

Listing 6.2: Hàm đánh giá mức độ đáp ứng sức chứa của sự kiện  
Dựa trên tỷ lệ Coverage, hàm phân loại kết quả thành ba mức:

- Thấp: tổng sức chứa nhỏ hơn 70% quy mô dự kiến.
- Gần đạt: tổng sức chứa từ 70% đến dưới 100% quy mô dự kiến.
- Đạt hoặc vượt: tổng sức chứa đạt hoặc vượt quy mô dự kiến.

## 6.2 Nhận xét

Hai hàm trên đều phục vụ việc tính toán dữ liệu từ các bảng đã có, không làm thay đổi dữ liệu trong cơ sở dữ liệu. Hàm `fn_Event_TotalConfirmedRentalFee` tập trung vào khía cạnh chi phí thuê của sự kiện, còn hàm `fn_Event_CapacityCoverageLabel` tập trung vào khả năng đáp ứng số lượng khách dự kiến.

Việc sử dụng con trỏ trong hai hàm giúp đáp ứng yêu cầu của bài tập về xử lý dữ liệu theo từng dòng. Mỗi hàm đều có kiểm tra tham số đầu vào, có truy vấn dữ liệu từ bảng `EVENT` và `SESSION`, có sử dụng vòng lặp để tính toán, sau đó trả về kết quả tương ứng.

## 6.3 Các hàm của phân hệ Khách hàng

Trong phân hệ phía Khách hàng, nhóm hiện thực hàm `fn_CalculateCustomerNetValue` nhằm tính toán giá trị ròng của một khách hàng trong một khoảng thời gian nhất định. Hàm này phục vụ cho việc đánh giá giá trị giao dịch thực tế của khách hàng sau khi đã trừ đi các khoản hoàn tiền phát sinh.

Hàm được hiện thực là:

- `fn_CalculateCustomerNetValue`: tính giá trị ròng của khách hàng dựa trên tổng tiền thanh toán thành công và tổng tiền hoàn lại.

Hàm có ba tham số đầu vào gồm mã khách hàng `@PersonID`, ngày bắt đầu `@FromDate` và ngày kết thúc `@ToDate`. Trước khi tính toán, hàm kiểm tra tính hợp lệ của các tham số đầu vào, kiểm tra khách hàng có tồn tại trong bảng `CUSTOMER` hay không và kiểm tra khách hàng có đơn hàng trong khoảng thời gian cần tính hay không.

Hàm sử dụng con trỏ để duyệt qua từng đơn hàng của khách hàng trong khoảng thời gian được truyền vào. Với mỗi đơn hàng, hàm tính tổng số tiền đã thanh toán thành công và tổng số tiền hoàn thực tế, sau đó cộng dồn để tính ra giá trị ròng cuối cùng.

### 6.3.1 Hàm tính giá trị ròng của khách hàng

Hàm `fn_CalculateCustomerNetValue` được dùng để tính giá trị ròng của một khách hàng trong hệ thống rạp chiếu phim. Giá trị ròng được hiểu là số tiền thực tế mà khách hàng đã chi tiêu sau khi trừ đi các khoản hoàn tiền được chấp nhận.

Công thức tính như sau:

$$\text{NetValue} = \text{TotalPaid} - \text{TotalRefund}$$

Trong đó:

- `TotalPaid`: tổng số tiền khách hàng đã thanh toán thành công.
- `TotalRefund`: tổng số tiền đã hoàn lại cho khách hàng.
- `NetValue`: giá trị ròng cuối cùng của khách hàng trong khoảng thời gian cần tính.

Tham số đầu vào của hàm gồm:

- @PersonID: mã khách hàng cần tính giá trị rỗng.
- @FromDate: ngày bắt đầu của khoảng thời gian tính toán.
- @ToDate: ngày kết thúc của khoảng thời gian tính toán.

Nếu mã khách hàng bị rỗng, ngày bắt đầu hoặc ngày kết thúc bị rỗng, hoặc ngày bắt đầu lớn hơn ngày kết thúc thì hàm trả về NULL. Nếu khách hàng không tồn tại trong bảng CUSTOMER, hàm cũng trả về NULL. Nếu khách hàng tồn tại nhưng không có đơn hàng nào trong khoảng thời gian được truyền vào, hàm trả về 0.

```
1 SET @PersonID = LTRIM(RTRIM(@PersonID));
2
3 IF @PersonID IS NULL OR @PersonID = ''
4     RETURN NULL;
5
6 IF @FromDate IS NULL OR @ToDate IS NULL
7     RETURN NULL;
8
9 IF @FromDate > @ToDate
10    RETURN NULL;
11
12 IF NOT EXISTS
13 (
14     SELECT 1
15     FROM CUSTOMER
16     WHERE PersonID = @PersonID
17 )
18     RETURN NULL;
19
20 IF NOT EXISTS
21 (
22     SELECT 1
23     FROM [ORDER]
24     WHERE PersonID = @PersonID
25           AND OrderDate BETWEEN @FromDate AND @ToDate
26 )
27     RETURN 0;
```

Listing 6.3: Kiểm tra tham số đầu vào trong hàm fn\_CalculateCustomerNetValue  
Sau khi kiểm tra dữ liệu đầu vào, hàm khai báo con trỏ cur\_Order để duyệt qua từng đơn hàng của khách hàng trong khoảng thời gian cần tính. Mỗi dòng dữ liệu được con trỏ duyệt qua tương ứng với một đơn hàng của khách hàng.

```
1 DECLARE cur_Order CURSOR LOCAL FORWARD_ONLY READ_ONLY FOR
2     SELECT OrderID
3     FROM [ORDER]
4     WHERE PersonID = @PersonID
5           AND OrderDate BETWEEN @FromDate AND @ToDate;
6
7 OPEN cur_Order;
8
9 FETCH NEXT FROM cur_Order INTO @CurrentOrderID;
10
11 WHILE @@FETCH_STATUS = 0
12 BEGIN
```

```
13  -- Tính toán trên từng đơn hàng
14  FETCH NEXT FROM cur_Order INTO @CurrentOrderID;
15  END
16
17  CLOSE cur_Order;
18  DEALLOCATE cur_Order;
```

Listing 6.4: Con trỏ duyệt từng đơn hàng của khách hàng

Với mỗi đơn hàng, hàm thực hiện hai bước tính toán chính. Thứ nhất, hàm tính tổng số tiền thanh toán thành công của đơn hàng từ bảng `PAYMENT_TRANSACTION`. Chỉ những giao dịch có trạng thái thanh toán là `success` mới được cộng vào tổng tiền đã thanh toán.

```
1  SELECT @PaidOfOrder = ISNULL(SUM(PT.PaymentTransactionAmount), 0)
2  FROM PAYMENT_TRANSACTION AS PT
3  WHERE PT.OrderID = @CurrentOrderID
4  AND LOWER(LTRIM(RTRIM(PT.PaymentTransactionPaymentStatus))) = N'success';
```

Listing 6.5: Tính tổng tiền thanh toán thành công của từng đơn hàng

Thứ hai, hàm tính tổng số tiền hoàn thực tế của đơn hàng. Khoản hoàn tiền được lấy từ bảng `REFUND_EXCHANGE_REQUEST` thông qua bảng `ORDER_DETAIL`. Hàm sử dụng `DISTINCT RequestID` để tránh cộng trùng một yêu cầu hoàn tiền nếu yêu cầu đó xuất hiện ở nhiều dòng chi tiết đơn hàng.

```
1  SELECT @RefundOfOrder = ISNULL(SUM(RER.RequestActualRefundAmount), 0)
2  FROM
3  (
4      SELECT DISTINCT OD.RequestID
5      FROM ORDER_DETAIL AS OD
6      WHERE OD.OrderID = @CurrentOrderID
7      AND OD.RequestID IS NOT NULL
8  ) AS X
9  INNER JOIN REFUND_EXCHANGE_REQUEST AS RER
10 ON RER.RequestID = X.RequestID
11 WHERE RER.RequestActualRefundAmount IS NOT NULL
12 AND LOWER(LTRIM(RTRIM(RER.RequestStatus))) IN
13 (N'completed', N'done', N'đã hoàn thành', N'hoàn thành', N'Approved');
```

Listing 6.6: Tính tổng tiền hoàn thực tế của từng đơn hàng

Sau khi tính được tiền thanh toán và tiền hoàn của từng đơn hàng, hàm cộng dồn hai giá trị này vào biến `@TotalPaid` và `@TotalRefund`. Khi con trỏ duyệt hết các đơn hàng, hàm tính giá trị ròng bằng cách lấy tổng tiền đã thanh toán trừ đi tổng tiền hoàn.

```
1  SET @TotalPaid = @TotalPaid + ISNULL(@PaidOfOrder, 0);
2  SET @TotalRefund = @TotalRefund + ISNULL(@RefundOfOrder, 0);
3
4  SET @NetValue = @TotalPaid - @TotalRefund;
5
6  IF @NetValue < 0
7      SET @NetValue = 0;
8
9  RETURN @NetValue;
```

Listing 6.7: Tính giá trị ròng cuối cùng của khách hàng

Trong trường hợp giá trị ròng nhỏ hơn 0, hàm ép kết quả về 0. Điều này giúp tránh trường hợp tổng tiền hoàn lớn hơn tổng tiền đã thanh toán, làm kết quả cuối cùng bị âm.

Hàm `fn_CalculateCustomerNetValue` có thể được sử dụng trong các chức năng thống kê khách hàng, ví dụ như xác định khách hàng có giá trị giao dịch cao, đánh giá mức độ đóng góp doanh thu của từng khách hàng hoặc hỗ trợ phân loại khách hàng theo giá trị mua hàng.

### 6.3.2 Nhận xét

Hàm `fn_CalculateCustomerNetValue` phục vụ việc tính toán dữ liệu từ các bảng đã có, không làm thay đổi dữ liệu trong cơ sở dữ liệu. Hàm lấy dữ liệu từ các bảng `CUSTOMER`, `ORDER`, `PAYMENT_TRANSACTION`, `ORDER_DETAIL` và `REFUND_EXCHANGE_REQUEST` để tính giá trị ròng của khách hàng trong một khoảng thời gian.

Hàm có kiểm tra tham số đầu vào, có truy vấn dữ liệu từ nhiều bảng, có sử dụng con trỏ để duyệt từng đơn hàng và có vòng lặp `WHILE` để cộng dồn kết quả. Vì vậy, hàm đáp ứng yêu cầu về việc sử dụng `IF`, sử dụng con trỏ, sử dụng vòng lặp, truy vấn dữ liệu và tính toán dữ liệu được lưu trữ.

Kết quả của hàm giúp hệ thống đánh giá chính xác hơn giá trị thực tế của khách hàng sau khi đã trừ các khoản hoàn tiền. Tuy nhiên, hàm này tính trên tổng tiền thanh toán của cả đơn hàng, nên số tiền có thể bao gồm vé xem phim, sản phẩm bán kèm, combo hoặc các khoản khác trong cùng đơn hàng. Nếu muốn tính riêng giá trị sản phẩm hoặc riêng giá trị vé, hệ thống cần xây dựng thêm hàm hoặc thủ tục có liên kết trực tiếp với bảng `PRODUCT_DETAIL` hoặc `TICKET_DETAIL`.

# CHƯƠNG 7. HIỆN THỰC ỨNG DỤNG

## 7.1 Tổng quan giao diện phía Rạp

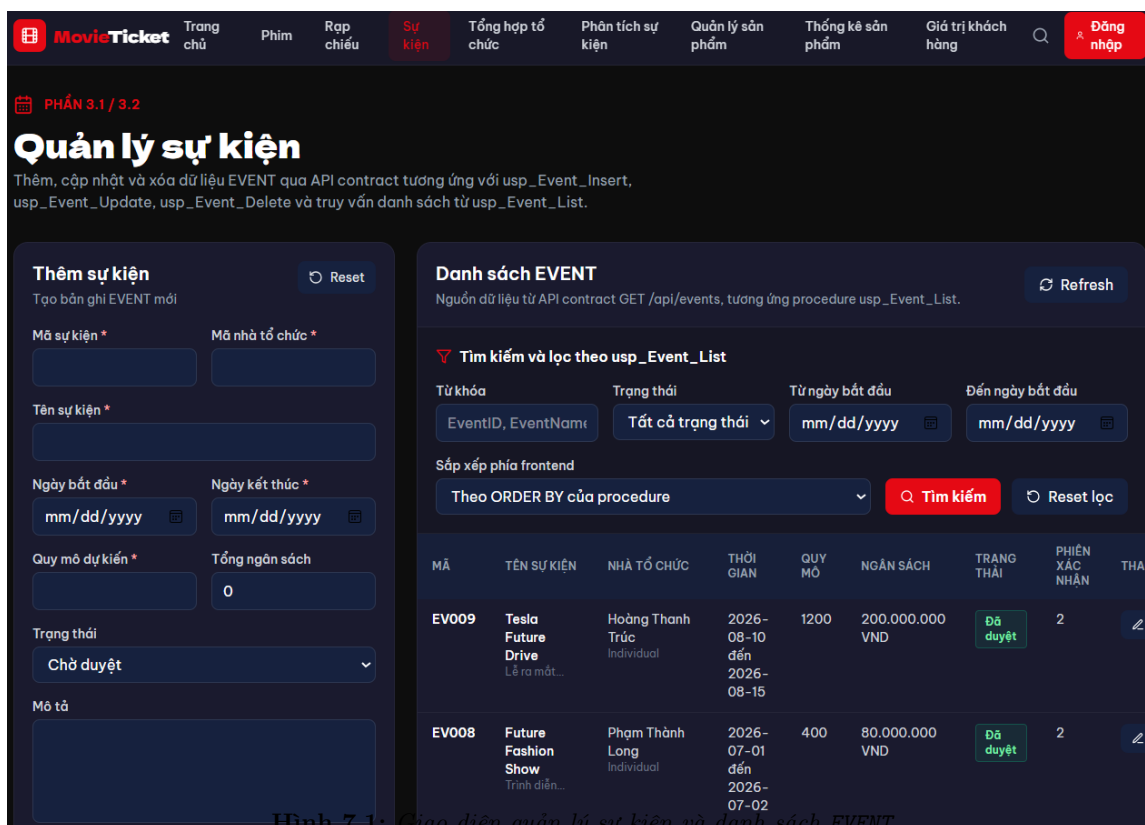
Ứng dụng được xây dựng để minh họa việc kết nối giữa giao diện người dùng, backend và cơ sở dữ liệu. Đối với phân hệ phía Rạp, các giao diện chính tập trung vào bảng **EVENT**, thủ tục truy vấn dữ liệu và các hàm tính toán liên quan đến sự kiện.

Các thao tác thêm, sửa, xóa sự kiện được thực hiện thông qua các thủ tục đã hiện thực ở câu 2.1. Các chức năng hiển thị danh sách, tìm kiếm, lọc và thống kê được thực hiện thông qua các thủ tục truy vấn ở câu 2.3. Ngoài ra, ứng dụng cũng có giao diện riêng để minh họa việc gọi các hàm ở câu 2.4.

### 7.1.1 Giao diện quản lý sự kiện

Giao diện quản lý sự kiện được dùng để thực hiện các thao tác trên bảng **EVENT**. Phần bên trái là form thêm sự kiện mới, tương ứng với thủ tục `usp_Event_Insert`. Phần bên phải hiển thị danh sách sự kiện từ thủ tục `usp_Event_List`. Người dùng có thể tìm kiếm theo từ khóa, lọc theo trạng thái, lọc theo khoảng ngày và chọn từng dòng dữ liệu để cập nhật hoặc xóa sự kiện.

Các thao tác cập nhật và xóa dữ liệu tương ứng với hai thủ tục `usp_Event_Update` và `usp_Event_Delete`. Khi dữ liệu nhập vào không hợp lệ hoặc thao tác không thỏa điều kiện nghiệp vụ, backend trả về thông báo lỗi để hiển thị trên giao diện.



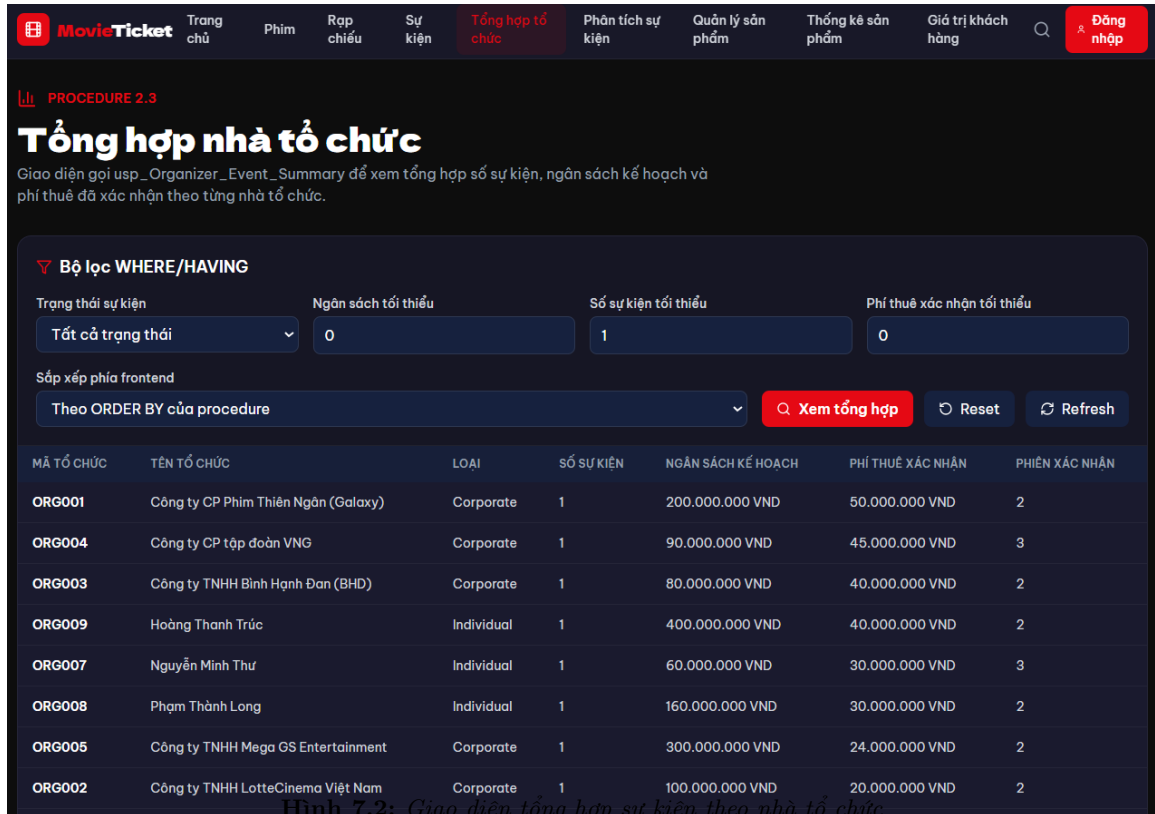
| MÃ    | TÊN SỰ KIỆN                          | NHÀ TỔ CHỨC                    | THỜI GIAN                 | QUY MÔ | NGÂN SÁCH       | TRANG THÁI | PHIÊN XÁC NHẬN | THA |
|-------|--------------------------------------|--------------------------------|---------------------------|--------|-----------------|------------|----------------|-----|
| EV009 | Tesla Future Drive<br>Lễ ra mắt...   | Hoàng Thanh Trúc<br>Individual | 2026-08-10 đến 2026-08-15 | 1200   | 200.000.000 VND | Đã duyệt   | 2              |     |
| EV008 | Future Fashion Show<br>Trình diễn... | Phạm Thành Long<br>Individual  | 2026-07-01 đến 2026-07-02 | 400    | 80.000.000 VND  | Đã duyệt   | 2              |     |

Hình 7.1: Giao diện quản lý sự kiện và danh sách EVENT

### 7.1.2 Giao diện tổng hợp nhà tổ chức

Giao diện tổng hợp nhà tổ chức được dùng để minh họa thủ tục truy vấn `usp_Organizer_Event_Summary`. Người dùng có thể lọc dữ liệu theo trạng thái sự kiện, ngân sách tối thiểu, số sự kiện tối thiểu và phí thuê xác nhận tối thiểu. Các tham số này tương ứng với các điều kiện `WHERE` và `HAVING` trong thủ tục.

Kết quả trả về cho biết số lượng sự kiện, tổng ngân sách kế hoạch và tổng phí thuê đã xác nhận theo từng nhà tổ chức. Giao diện này phục vụ việc thống kê và theo dõi hoạt động tổ chức sự kiện tại rạp.



**Tổng hợp nhà tổ chức**

Giao diện gọi `usp_Organizer_Event_Summary` để xem tổng hợp số sự kiện, ngân sách kế hoạch và phí thuê đã xác nhận theo từng nhà tổ chức.

**Bộ lọc WHERE/HAVING**

Trạng thái sự kiện: Tất cả trạng thái | Ngân sách tối thiểu: 0 | Số sự kiện tối thiểu: 1 | Phí thuê xác nhận tối thiểu: 0

Sắp xếp phía frontend: Theo ORDER BY của procedure | Xem tổng hợp | Reset | Refresh

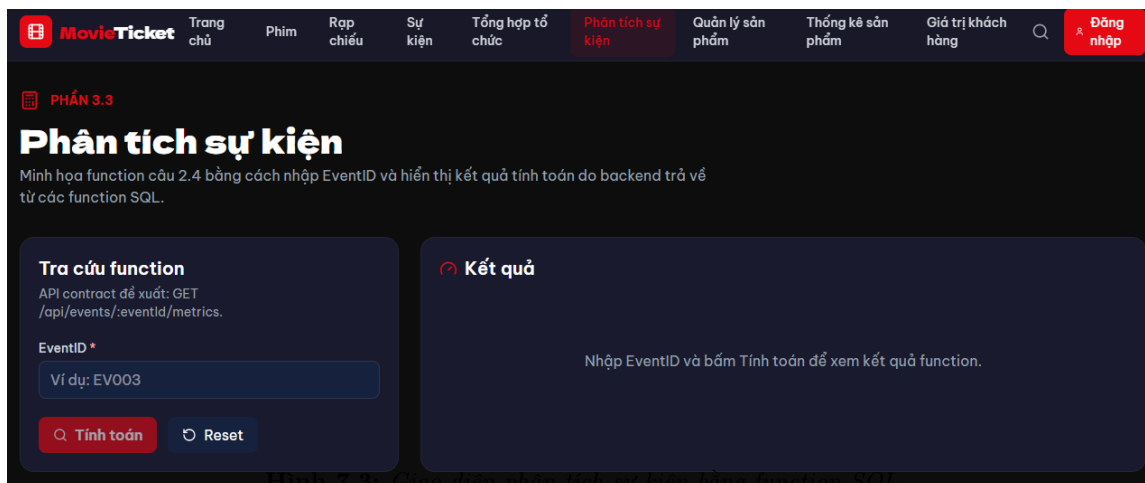
| MÃ TỔ CHỨC | TÊN TỔ CHỨC                         | LOẠI       | SỐ SỰ KIỆN | NGÂN SÁCH KẾ HOẠCH | PHÍ THUÊ XÁC NHẬN | PHIÊN XÁC NHẬN |
|------------|-------------------------------------|------------|------------|--------------------|-------------------|----------------|
| ORG001     | Công ty CP Phim Thiên Ngân (Galaxy) | Corporate  | 1          | 200.000.000 VND    | 50.000.000 VND    | 2              |
| ORG004     | Công ty CP tập đoàn VNG             | Corporate  | 1          | 90.000.000 VND     | 45.000.000 VND    | 3              |
| ORG003     | Công ty TNHH Bình Hạnh Đan (BHD)    | Corporate  | 1          | 80.000.000 VND     | 40.000.000 VND    | 2              |
| ORG009     | Hoàng Thanh Trúc                    | Individual | 1          | 400.000.000 VND    | 40.000.000 VND    | 2              |
| ORG007     | Nguyễn Minh Thư                     | Individual | 1          | 60.000.000 VND     | 30.000.000 VND    | 3              |
| ORG008     | Phạm Thành Long                     | Individual | 1          | 160.000.000 VND    | 30.000.000 VND    | 2              |
| ORG005     | Công ty TNHH Mega GS Entertainment  | Corporate  | 1          | 300.000.000 VND    | 24.000.000 VND    | 2              |
| ORG002     | Công ty TNHH LotteCinema Việt Nam   | Corporate  | 1          | 100.000.000 VND    | 20.000.000 VND    | 2              |

Hình 7.2: Giao diện tổng hợp sự kiện theo nhà tổ chức

### 7.1.3 Giao diện phân tích sự kiện

Giao diện phân tích sự kiện được dùng để minh họa việc gọi các hàm ở câu 2.4. Người dùng nhập mã sự kiện `EventID`, sau đó backend gọi các hàm SQL để trả về kết quả tính toán.

Hai hàm được minh họa trên giao diện gồm `fn_Event_CapacityCoverageLabel` và `fn_Event_TotalConfirmedRentalFee`. Hàm thứ nhất đánh giá mức độ bao phủ sức chứa của các phiên sự kiện so với quy mô dự kiến. Hàm thứ hai tính tổng phí thuê của các phiên sự kiện đã được xác nhận.



Hình 7.3: Giao diện phân tích sự kiện bằng function SQL

## 7.2 Hiện thực ứng dụng phía Khách hàng và Sản phẩm

Bên cạnh phân hệ phía Rạp, nhóm cũng hiện thực các giao diện phục vụ nghiệp vụ quản lý sản phẩm và phân tích giá trị khách hàng. Các giao diện này được xây dựng để minh họa việc kết nối giữa frontend, backend và cơ sở dữ liệu. Thông qua giao diện, người dùng có thể thao tác dữ liệu sản phẩm, xem thống kê doanh thu sản phẩm và tra cứu giá trị ròng của khách hàng theo khoảng thời gian.

Phần hiện thực phía Khách hàng và Sản phẩm gồm ba giao diện chính:

- Giao diện quản lý sản phẩm, dùng để thêm, cập nhật, xóa và hiển thị danh sách sản phẩm.
- Giao diện thống kê sản phẩm, dùng để hiển thị kết quả tổng hợp từ thủ tục truy vấn dữ liệu.
- Giao diện giá trị khách hàng, dùng để minh họa kết quả tính toán từ hàm SQL.

### 7.2.1 Giao diện quản lý sản phẩm

Giao diện quản lý sản phẩm được dùng để thực hiện các thao tác trên bảng **PRODUCT**. Các chức năng thêm, cập nhật và xóa sản phẩm tương ứng với các thủ tục **sp\_InsertProduct**, **sp\_UpdateProduct** và **sp\_DeleteProduct**. Danh sách sản phẩm được hiển thị ở phần bên phải giao diện, giúp người dùng theo dõi dữ liệu hiện có trong hệ thống.

Trên giao diện, người dùng có thể nhập các thông tin cơ bản của sản phẩm như mã sản phẩm, tên sản phẩm, loại sản phẩm, giá cơ bản và trạng thái sản phẩm. Các trường dữ liệu quan trọng được đánh dấu bắt buộc để hạn chế việc gửi dữ liệu thiếu thông tin xuống backend. Khi người dùng thực hiện thao tác thêm, sửa hoặc xóa, frontend gửi request đến backend, sau đó backend gọi thủ tục SQL tương ứng để thao tác với cơ sở dữ liệu.

Giao diện này đáp ứng yêu cầu hiện thực chức năng thêm, sửa và xóa dữ liệu vào bảng được chọn. Đồng thời, việc hiển thị danh sách sản phẩm giúp người dùng kiểm tra lại kết quả sau mỗi thao tác.

PRODUCT - PHẦN 3.1

Quản lý sản phẩm

Thêm, cập nhật và xóa PRODUCT qua API contract tương ứng với sp\_InsertProduct, sp\_UpdateProduct và sp\_DeleteProduct.

Thêm sản phẩm

Reset

Tạo bản ghi PRODUCT mới

ProductID \*

ProductName \*

ProductType

BasePrice \*

ProductStatus

Đang bán

Khi xóa, SQL chỉ cho phép xóa sản phẩm có trạng thái Ngừng kinh doanh và không phát sinh đơn hàng trong ngày hiện tại.

Thêm sản phẩm

Danh sách PRODUCT

Nguồn dữ liệu từ API contract GET /api/products.

Tìm mã, tên, logi...

Refresh

| PRODUCTID | PRODUCTNAME               | PRODUCTTYPE | BASEPRICE   | PRODUCTSTATUS    | THAO TÁC                      |
|-----------|---------------------------|-------------|-------------|------------------|-------------------------------|
| PR001     | Combo bắp nước tiêu chuẩn | Combo       | 135.000 VND | Đang bán         | <div>Sửa</div> <div>Xóa</div> |
| PR002     | Nước ngọt                 | Nước        | 55.000 VND  | Đang bán         | <div>Sửa</div> <div>Xóa</div> |
| PR003     | Nước ngọt size lớn        | Nước        | 60.000 VND  | Đang bán         | <div>Sửa</div> <div>Xóa</div> |
| PR004     | Bắp rang bơ               | Bắp         | 45.000 VND  | Đang bán         | <div>Sửa</div> <div>Xóa</div> |
| PR005     | Nước suối                 | Nước        | 30.000 VND  | Đang bán         | <div>Sửa</div> <div>Xóa</div> |
| PR014     | Trà đào                   | Nước        | 45.000 VND  | Ngừng kinh doanh | <div>Sửa</div> <div>Xóa</div> |
| PR015     | Nước cam                  | Nước        | 50.000 VND  | Ngừng kinh doanh | <div>Sửa</div> <div>Xóa</div> |

Hình 7.4: Giao diện quản lý sản phẩm

### 7.2.2 Giao diện thống kê sản phẩm

Giao diện thống kê sản phẩm được dùng để minh họa thủ tục truy vấn sp\_GetProductSalesSummary. Đây là thủ tục dùng để tổng hợp dữ liệu bán hàng của sản phẩm dựa trên các điều kiện lọc do người dùng nhập vào.

Các tham số trên giao diện gồm loại sản phẩm, ngày bắt đầu, ngày kết thúc, số lượng bán tối thiểu và doanh thu tối thiểu. Các tham số này được truyền xuống backend và ánh xạ vào các tham số của thủ tục SQL. Kết quả trả về được dùng để hiển thị các chỉ số tổng hợp như số sản phẩm tìm thấy, tổng số lượng bán và tổng doanh thu.

Giao diện này phục vụ yêu cầu hiển thị dữ liệu từ thủ tục truy vấn. Người dùng có thể thay đổi điều kiện lọc để xem kết quả thống kê theo từng nhóm sản phẩm hoặc theo từng khoảng thời gian khác nhau. Ngoài ra, giao diện còn hỗ trợ tìm kiếm và sắp xếp ở phía frontend để việc quan sát dữ liệu thuận tiện hơn.

PRODUCT - PHẦN 3.2

Thống kê sản phẩm

Hiển thị dữ liệu từ procedure sp\_GetProductSalesSummary với filter theo ProductType, khoảng ngày bán, ngưỡng số lượng và ngưỡng doanh thu.

Bộ lọc

ProductType

Tất cả

FromDate

mm/dd/yyyy

ToDate

mm/dd/yyyy

SẢN PHẨM TÌM THẤY

0

TỔNG SỐ LƯỢNG BÁN

0

TỔNG DOANH THU

0 VND

Kết quả thống kê

sp\_GetProductSalesSummary

API contract: GET /api/products/sales-summary.

Tìm mã, tên

Theo ORDER BY procedu

Refresh

Hình 7.5: Giao diện thống kê sản phẩm

### 7.2.3 Giao diện giá trị khách hàng

Giao diện giá trị khách hàng được dùng để minh họa hàm `fn_CalculateCustomerNetValue`. Hàm này nhận vào mã khách hàng và khoảng thời gian cần tính toán, sau đó trả về giá trị ròng của khách hàng trong khoảng thời gian đó.

Trên giao diện, người dùng nhập `PersonID`, `FromDate` và `ToDate`. Sau khi người dùng bấm nút tính giá trị, frontend gửi các tham số này đến backend. Backend tiếp tục gọi hàm SQL `fn_CalculateCustomerNetValue` và trả kết quả về cho giao diện.

Kết quả trả về là giá trị `NetValue`, được dùng để đánh giá mức đóng góp của khách hàng dựa trên dữ liệu giao dịch trong cơ sở dữ liệu. Giao diện này đáp ứng yêu cầu minh họa việc gọi hàm SQL có tham số đầu vào thông qua ứng dụng.

**PRODUCT - PHẦN 3.3**

## Giá trị khách hàng

Minh họa function `dbo.fn_CalculateCustomerNetValue` với `PersonID`, `FromDate` và `ToDate`. Kết quả chính là `NetValue` dạng `DECIMAL(18,2)`.

**Tham số function**  
API contract: GET  
`/api/customers/net-value`.

**PersonID \***  
VD: PER001

**FromDate \***  
mm/dd/yyyy

**ToDate \***  
mm/dd/yyyy

**Kết quả NetValue**  
Nguồn dữ liệu: `dbo.fn_CalculateCustomerNetValue`.

**Chưa có kết quả**  
Nhập `PersonID`, `FromDate`, `ToDate` rồi bấm Tính giá trị.

Hình 7.6: Giao diện tính giá trị khách hàng

### 7.2.4 Nhận xét

Các giao diện phía Khách hàng và Sản phẩm cho thấy ứng dụng đã kết nối với cơ sở dữ liệu thông qua backend. Giao diện quản lý sản phẩm minh họa các thao tác thêm, sửa và xóa dữ liệu. Giao diện thống kê sản phẩm minh họa việc gọi thủ tục truy vấn dữ liệu có tham số lọc. Giao diện giá trị khách hàng minh họa việc gọi hàm SQL để tính toán dữ liệu theo tham số đầu vào.

Như vậy, phần hiện thực ứng dụng đã bao phủ các yêu cầu chính của đề bài: thao tác dữ liệu bằng thủ tục, hiển thị dữ liệu từ thủ tục truy vấn và minh họa kết quả trả về từ hàm trong cơ sở dữ liệu.

# CHƯƠNG 8. TESTING VÀ DEMO

## 8.1 Mục tiêu kiểm thử

Phần kiểm thử được thực hiện để kiểm tra các chức năng đã hiện thực cho hệ thống rạp phim, bao gồm thủ tục thêm, sửa, xóa dữ liệu trên bảng **EVENT** và **PRODUCT**, trigger trên các bảng, các thủ tục truy vấn dữ liệu, các hàm tính toán và giao diện ứng dụng.

Các kịch bản kiểm thử được chuẩn bị theo hai nhóm chính. Nhóm thứ nhất là kiểm thử trực tiếp bằng câu lệnh SQL để kiểm tra procedure, trigger và function. Nhóm thứ hai là kiểm thử thông qua giao diện ứng dụng để kiểm tra việc kết nối giữa frontend, backend và cơ sở dữ liệu.

## 8.2 Kiểm thử bằng câu lệnh SQL

Các câu lệnh SQL được dùng để kiểm tra trực tiếp những thành phần đã hiện thực trong cơ sở dữ liệu. Nhóm không trình bày toàn bộ script kiểm thử trong báo cáo, mà chỉ liệt kê các tình huống chính. Các câu lệnh chi tiết được chuẩn bị trong file SQL demo để thực thi khi báo cáo.

| STT | Nội dung kiểm thử                                                  | Kết quả mong đợi                                               | Thành phần kiểm thử                             |
|-----|--------------------------------------------------------------------|----------------------------------------------------------------|-------------------------------------------------|
| 1   | Thêm sự kiện mới với dữ liệu hợp lệ                                | Sự kiện được thêm vào bảng <b>EVENT</b>                        | <b>usp_Event_Insert</b>                         |
| 2   | Thêm sự kiện có mã nhà tổ chức không tồn tại                       | Thủ tục báo lỗi và không thêm dữ liệu                          | <b>usp_Event_Insert</b>                         |
| 3   | Cập nhật thời gian sự kiện không bao phủ các phiên đã có           | Thủ tục báo lỗi và không cập nhật dữ liệu                      | <b>usp_Event_Update</b>                         |
| 4   | Xóa sự kiện đã phát sinh phiên tổ chức                             | Thủ tục báo lỗi và không xóa sự kiện                           | <b>usp_Event_Delete</b>                         |
| 5   | Thêm phiên sự kiện nằm ngoài thời gian của sự kiện                 | Trigger báo lỗi và rollback thao tác                           | <b>TRG_Session_Inside_Event_Date</b>            |
| 6   | Thêm, sửa hoặc xóa phiên có trạng thái đã xác nhận                 | Số phiên đã xác nhận của sự kiện được cập nhật lại             | <b>TRG_Session_Update_ConfirmedSessionCount</b> |
| 7   | Truy vấn danh sách sự kiện theo từ khóa, trạng thái và khoảng ngày | Trả về đúng các sự kiện thỏa điều kiện lọc                     | <b>usp_Event_List</b>                           |
| 8   | Thống kê sự kiện theo nhà tổ chức                                  | Trả về số sự kiện, ngân sách và phí thuê theo từng nhà tổ chức | <b>usp_Organizer_Event_Summary</b>              |
| 9   | Tính tổng phí thuê đã xác nhận của một sự kiện                     | Trả về tổng phí thuê của các phiên đã xác nhận                 | <b>fn_Event_TotalConfirmedRentalFee</b>         |
| 10  | Đánh giá mức độ đáp ứng sức chứa của sự kiện                       | Trả về nhãn đánh giá tương ứng                                 | <b>fn_Event_CapacityCoverageLabel</b>           |

**Bảng 8.1:** Các kịch bản kiểm thử SQL chính

| STT | Nội dung kiểm thử                                                               | Kết quả mong đợi                                                                                       | Thành phần kiểm thử                        |
|-----|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------|
| 11  | Thêm sản phẩm mới với dữ liệu hợp lệ                                            | Sản phẩm được thêm thành công vào bảng <b>PRODUCT</b>                                                  | sp_InsertProduct                           |
| 12  | Cập nhật giá bán của sản phẩm đang tồn tại                                      | Thông tin sản phẩm được cập nhật thành công trong bảng <b>PRODUCT</b>                                  | sp_UpdateProduct                           |
| 13  | Xóa sản phẩm đã ngừng kinh doanh và chưa phát sinh đơn hàng trong ngày          | Sản phẩm được xóa thành công khỏi bảng <b>PRODUCT</b>                                                  | sp_DeleteProduct                           |
| 14  | Truy vấn thống kê doanh số sản phẩm theo loại sản phẩm và khoảng thời gian      | Trả về danh sách sản phẩm có phát sinh bán, kèm số lượng bán và doanh thu tương ứng                    | sp_GetProductSalesSummary                  |
| 15  | Tính giá trị ròng của khách hàng trong một khoảng thời gian                     | Trả về giá trị ròng bằng tổng tiền đã thanh toán trừ tổng tiền hoàn                                    | fn_CalculateCustomerNetValue               |
| 16  | Cập nhật thay đổi người viết review sau khi review đã được tạo                  | Trigger báo lỗi và không cho thay đổi <b>PersonID</b> của review                                       | trg_Review PreventUpdatePersonID           |
| 17  | Gửi đánh giá sản phẩm mà khách hàng chưa từng mua                               | Trigger báo lỗi và không cho thêm đánh giá sản phẩm                                                    | trg_ProductReview OnlyPurchasedProduct     |
| 18  | Thêm lịch sử điểm cho khách hàng thành viên                                     | <b>CurrentPoints</b> của khách hàng được cập nhật lại theo tổng <b>PointChange</b>                     | trg_PointHistory UpdateCurrentPoints       |
| 19  | Áp dụng voucher không còn hiệu lực hoặc đã hết hạn                              | Trigger báo lỗi và rollback thao tác trên bảng <b>APPLY</b>                                            | trg_Check_Apply_Rules                      |
| 20  | Áp dụng voucher làm tổng giảm giá vượt quá giá trị gốc của đơn hàng             | Trigger báo lỗi và không cho lưu mức giảm giá không hợp lệ                                             | trg_Check_Apply_Rules                      |
| 21  | Ghi lịch sử điểm cho khách vắng lai                                             | Trigger báo lỗi vì khách vắng lai không được tích điểm hoặc lưu lịch sử điểm                           | trg_Check_GuestCustomer NoPoints           |
| 22  | Cập nhật vé sang trạng thái đã check-in nhưng thiếu thời gian check-in          | Trigger báo lỗi và không cho cập nhật vé                                                               | trg_Check_ETicket OneCheckinOnly           |
| 23  | Check-in lại một vé đã được check-in thành công trước đó                        | Trigger báo lỗi vì mỗi vé điện tử chỉ được check-in một lần                                            | trg_Check_ETicket OneCheckinOnly           |
| 24  | Gửi đánh giá đơn hàng bởi người không phải chủ đơn hàng                         | Trigger báo lỗi và không cho thêm đánh giá đơn hàng                                                    | trg_Check_OrderReview Condition            |
| 25  | Gửi đánh giá sản phẩm mà khách hàng chưa từng mua trong đơn hàng của chính mình | Trigger báo lỗi và không cho thêm đánh giá sản phẩm                                                    | trg_Check_ProductReview Condition          |
| 26  | Tạo yêu cầu hoàn, đổi hoặc hủy cho vé đã check-in                               | Trigger báo lỗi vì vé đã check-in không được hoàn, đổi hoặc hủy                                        | trg_Check_RefundExchange NoCheckedInTicket |
| 27  | Thêm, sửa hoặc xóa chi tiết đơn hàng                                            | Tổng tiền đơn hàng trong bảng <b>ORDER</b> được cập nhật lại theo chi tiết đơn hàng và voucher áp dụng | trg_UpdateOrderTotal FromOrderDetail       |
| 28  | Thêm, sửa hoặc xóa voucher áp dụng cho đơn hàng                                 | Tổng tiền đơn hàng được cập nhật lại sau khi tính tổng giảm giá                                        | trg_UpdateOrderTotal FromApply             |
| 29  | Thêm, sửa hoặc xóa lịch sử điểm của khách hàng thành viên                       | <b>REGISTERED_CUSTOMER.CurrentPoints</b> được cập nhật bằng tổng <b>PointChange</b> hiện có            | trg_UpdateRegistered CustomerCurrentPoints |

**Bảng 8.2:** Các kịch bản kiểm thử SQL chính (phía khách hàng và sản phẩm)

## 8.3 Kiểm thử thông qua giao diện ứng dụng

Bên cạnh việc kiểm thử trực tiếp bằng SQL, nhóm cũng kiểm thử thông qua giao diện ứng dụng. Mục tiêu của phần này là kiểm tra ứng dụng có gọi đúng backend và backend có thực thi đúng các procedure hoặc function trong cơ sở dữ liệu hay không.

Các giao diện được dùng để demo gồm:

- Giao diện quản lý sự kiện: dùng để thêm, sửa, xóa và hiển thị danh sách sự kiện.
- Giao diện tổng hợp nhà tổ chức: dùng để gọi thủ tục thống kê `usp_Organizer_Event_Summary`.
- Giao diện phân tích sự kiện: dùng để gọi các hàm `fn_Event_TotalConfirmedRentalFee` và `fn_Event_CapacityCoverageLabel`.
- Giao diện quản lý sản phẩm: dùng để thêm, sửa, xóa và hiển thị danh sách sản phẩm.
- Giao diện thống kê sản phẩm: dùng để gọi thủ tục thống kê `sp_GetProductSalesSummary`.
- Giao diện giá trị khách hàng: dùng để gọi hàm `fn_CalculateCustomerNetValue`.

Khi demo, nhóm sẽ nhập dữ liệu trực tiếp trên giao diện, sau đó kiểm tra kết quả trả về. Đối với các trường hợp dữ liệu không hợp lệ, hệ thống cần hiển thị thông báo lỗi tương ứng thay vì ghi dữ liệu sai vào cơ sở dữ liệu.

# CHƯƠNG 9. KẾT QUẢ VÀ KẾT LUẬN

## 9.1 Kết quả

Sau quá trình thực hiện bài tập lớn số 2 môn Hệ cơ sở dữ liệu, nhóm 4 đã hoàn thành việc hiện thực hệ thống quản lý rạp chiếu phim dựa trên kết quả thiết kế ở bài tập lớn số 1. Hệ thống được xây dựng với hai phân hệ chính là phía Rạp và phía Khách hàng, bao gồm các chức năng quản lý rạp, phòng chiếu, sự kiện, khách hàng, đơn hàng, sản phẩm, vé, thanh toán, voucher, đánh giá và điểm tích lũy.

Về cơ sở dữ liệu, nhóm đã tạo các bảng cần thiết cho hệ thống, cài đặt khóa chính, khóa ngoại và các ràng buộc dữ liệu nhằm đảm bảo tính toàn vẹn. Dữ liệu mẫu cũng được chuẩn bị cho các bảng để phục vụ quá trình kiểm thử và demo. Các bảng dữ liệu được tổ chức theo từng nhóm nghiệp vụ rõ ràng, giúp hệ thống dễ quản lý và thuận tiện cho việc mở rộng sau này.

Về thủ tục thêm, sửa và xóa dữ liệu, nhóm đã xây dựng các stored procedure cho những bảng đại diện trong hệ thống. Các thủ tục này có kiểm tra dữ liệu đầu vào, kiểm tra điều kiện nghiệp vụ và trả về thông báo lỗi cụ thể khi dữ liệu không hợp lệ. Nhờ đó, các thao tác xử lý dữ liệu được thực hiện an toàn hơn và hạn chế sai sót trong quá trình nhập liệu.

Về trigger, nhóm đã hiện thực các trigger kiểm tra ràng buộc nghiệp vụ và cập nhật thuộc tính dẫn xuất. Ở phân hệ phía Rạp, trigger được dùng để kiểm tra thời gian phiên sự kiện và cập nhật số phiên đã xác nhận. Ở phân hệ phía Khách hàng, trigger được dùng để kiểm tra điều kiện đánh giá, áp dụng voucher, kiểm soát vé đã check-in, cập nhật tổng tiền đơn hàng và cập nhật điểm hiện tại của khách hàng. Các trigger này giúp hệ thống tự động kiểm soát dữ liệu khi có thao tác thêm, sửa hoặc xóa phát sinh.

Về thủ tục truy vấn dữ liệu, nhóm đã xây dựng các thủ tục phục vụ việc hiển thị danh sách, tìm kiếm, lọc và thống kê dữ liệu. Các thủ tục có sử dụng liên kết nhiều bảng, mệnh đề **WHERE**, **ORDER BY**, hàm tổng hợp, **GROUP BY** và **HAVING**. Kết quả truy vấn hỗ trợ việc theo dõi hoạt động của rạp, thống kê sự kiện, sản phẩm và doanh thu.

Về hàm, nhóm đã xây dựng các hàm phục vụ tính toán dữ liệu trong hệ thống. Các hàm có kiểm tra tham số đầu vào, truy vấn dữ liệu từ nhiều bảng liên quan, sử dụng con trỏ và vòng lặp để xử lý dữ liệu. Kết quả của các hàm hỗ trợ việc tính tổng phí thuê sự kiện, đánh giá khả năng đáp ứng sức chứa và tính giá trị ròng của khách hàng.

Ngoài ra, nhóm đã xây dựng ứng dụng minh họa có kết nối trực tiếp với cơ sở dữ liệu. Ứng dụng hỗ trợ các chức năng cơ bản như hiển thị danh sách, thêm, sửa, xóa dữ liệu, tìm kiếm, lọc và gọi các thủ tục hoặc hàm đã xây dựng. Thông qua quá trình kiểm thử, nhóm đã kiểm tra được các trường hợp dữ liệu hợp lệ và không hợp lệ, từ đó đánh giá hoạt động của các thủ tục, trigger, hàm và ràng buộc trong hệ thống.

Nhìn chung, nhóm đã hoàn thành các nội dung chính của bài tập lớn số 2, bao gồm hiện thực cơ sở dữ liệu, tạo dữ liệu mẫu, xây dựng thủ tục, trigger, hàm, ứng dụng minh họa và kiểm thử các chức năng chính của hệ thống.

## 9.2 Kết luận

Bài tập lớn số 2 đã giúp nhóm 4, học kỳ 252, Trường Đại học Bách khoa – Đại học Quốc gia Thành phố Hồ Chí Minh, vận dụng các kiến thức đã học trong môn Hệ cơ sở dữ liệu vào bài toán thực tế là hệ thống quản lý rạp chiếu phim.

Thông qua quá trình thực hiện đề tài, nhóm đã hiểu rõ hơn quy trình xây dựng một hệ thống cơ sở dữ liệu hoàn chỉnh, từ việc tạo bảng, thiết lập ràng buộc, viết stored procedure, trigger, function cho đến kết nối cơ sở dữ liệu với ứng dụng. Đề tài cũng giúp nhóm nhận thấy vai trò quan trọng của cơ sở dữ liệu trong việc lưu trữ, xử lý và đảm bảo tính nhất quán của dữ liệu trong các hệ thống quản lý.

Hệ thống đã đáp ứng được các chức năng cơ bản như quản lý dữ liệu rạp, sự kiện, khách hàng, đơn hàng, sản phẩm, vé, thanh toán, voucher, đánh giá và điểm tích lũy. Các thủ tục, trigger và hàm được xây dựng góp phần kiểm soát dữ liệu tốt hơn, hạn chế sai sót và hỗ trợ xử lý các nghiệp vụ phức tạp.

Tuy nhiên, do giới hạn về thời gian và phạm vi môn học, hệ thống vẫn còn một số hạn chế. Ứng dụng mới dừng ở mức minh họa các chức năng chính, chưa tối ưu hoàn toàn về giao diện và trải nghiệm người dùng. Một số chức năng nâng cao như thanh toán online thực tế, gửi email xác nhận, phân quyền chi tiết, tích hợp mã QR check-in thực tế hoặc quản lý nhiều chi nhánh rạp vẫn chưa được triển khai đầy đủ.

Trong tương lai, nhóm dự tính phát triển hệ thống để tối ưu các chức năng để có thể mang đến 1 sản phẩm có ích cho xã hội người tiêu dùng cũng như rèn luyện thêm về mặt kiến thức và kinh nghiệm cho bản thân là sinh viên của Trường Đại học Bách Khoa - ĐHQG Thành phố Hồ Chí Minh. Nhóm rất biết ơn và mong sẽ tiếp tục được thầy, cô và các giảng viên hỗ trợ thực hiện mục tiêu.

Tổng kết lại, bài tập lớn này đã giúp nhóm có thêm kinh nghiệm trong việc phân tích, hiện thực và kiểm thử một hệ thống cơ sở dữ liệu có tính ứng dụng thực tế. Đây là nền tảng quan trọng để nhóm tiếp tục phát triển kỹ năng thiết kế cơ sở dữ liệu, lập trình SQL và xây dựng ứng dụng trong các môn học cũng như dự án sau này.